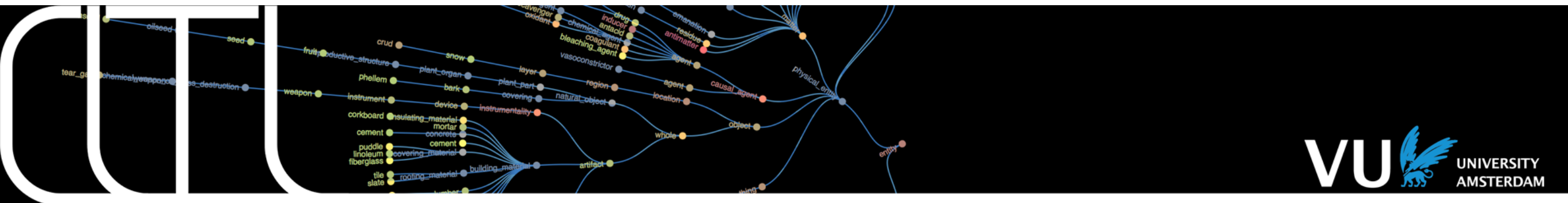


Text Mining 2026



Lecture 3: Machine Learning for NLP Ilia Markov



Overview

- Part I: NLP Pipelines
- Part II: Bag-of-Words classification
- Part III: Evaluation
- Part IV: Sequence classification
- Part V: Word Embeddings
- Part VI: Contextual Embeddings

Part III

Evaluation

How to trust data systems?

- **Evaluation metrics:** methods for measuring a system's performance during testing
- **Evaluation framework:** how to design an experiment to ensure the reliability of your metrics
- **Application perspective:** behavior in relation to the application (e.g., more precise or robust)

Evaluation metrics: Precision, Recall, F1-measure

$$\text{Precision} = \frac{tp}{tp + fp}$$

$$\text{Recall} = \frac{tp}{tp + fn}$$

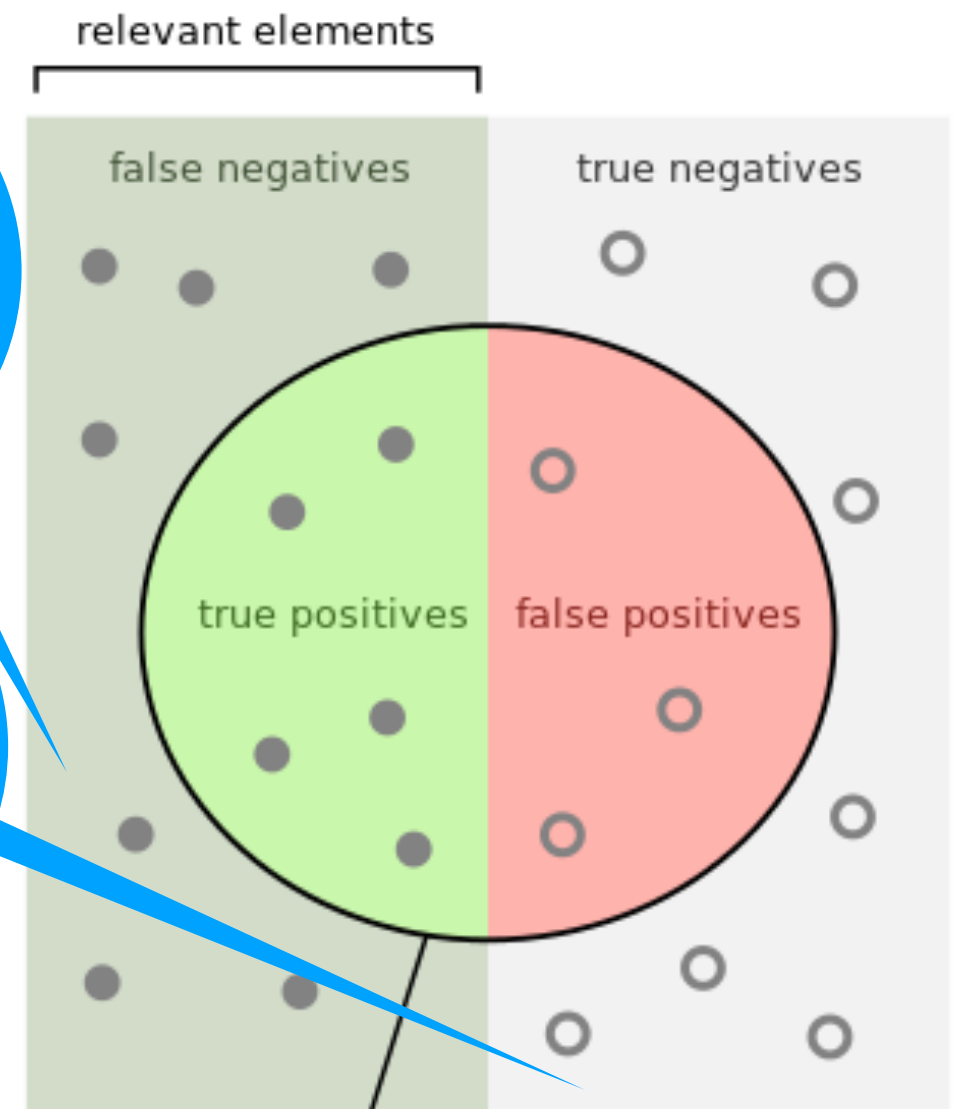
$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

correct cases
that a system
should have
found

cases that did
not belong to your
category

everything that
was selected by
the system

all the cases that
it should have
found



selected elements

How many selected
items are relevant?

$$\text{Precision} = \frac{\text{green semi-circle}}{\text{green semi-circle} + \text{red semi-circle}}$$

How many relevant
items are selected?

$$\text{Recall} = \frac{\text{green semi-circle}}{\text{green semi-circle} + \text{green rectangle}}$$

What is the simplest system that would achieve
100% recall in spam detection?

Contingency table (confusion matrix)

		<i>gold standard labels</i>		
		gold spam	gold no spam	
<i>system output labels</i>	system spam	true positive	false positive	$\text{precision} = \frac{tp}{tp+fp}$
	system no spam	false negative	true negative	
		$\text{recall} = \frac{tp}{tp+fn}$		$\text{accuracy} = \frac{tp+tn}{tp+fp+tn+fn}$

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Figure 6.4 Contingency table

Binary classification
accuracy, precision, recall, harmonic mean (F1)

If most cases are not spam and a system predicts everything as 'not spam,' it will have high accuracy. **Accuracy is a reliable metric for balanced data!**

Contingency table

		<i>gold standard labels</i>		$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$
		gold spam	gold no spam	
<i>system output labels</i>	system spam	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system no spam	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Figure 6.4 Contingency table

Gold data: 100 mails, 20 spam mails
System: detects 10 spam mails, 5 correct

5 TP + 75 TN

= 0.8 accuracy

5 TP + 5 FP + 75 TN + 15 FN

(see also lab sessions)

$$\frac{5 \text{ TP}}{5 \text{ TP} + 5 \text{ FP}} = 0.5 \text{ precision}$$

$$\frac{5 \text{ TP}}{5 \text{ TP} + 15 \text{ FN}} = 0.25 \text{ recall}$$

$$\frac{2 \times 0.5 \times 0.25}{0.5 + 0.25} = 0.33 \text{ F1}$$

Multiple classes: spam, urgent, normal

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	precision_u = $\frac{8}{8+10+1}$
	normal	5	60	50	precision_n = $\frac{60}{5+60+50}$
	spam	3	30	200	precision_s = $\frac{200}{3+30+200}$
		recall_u = $\frac{8}{8+5+3}$	recall_n = $\frac{60}{10+60+30}$	recall_s = $\frac{200}{1+50+200}$	

Figure 6.5 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2) , how many documents from c_1 were (in)correctly assigned to c_2

- 3 classes = more complex (FPs are distributed over 2 classes) -> calculate, e.g., precision for each category and then average
- **one-of** or multinomial: each instance belongs to exactly **one** class out of multiple possible categories
- **any-of**: an instance can belong to **multiple** categories simultaneously

Micro & Macro averaging

Class 1: Urgent			Class 2: Normal			Class 3: Spam			Pooled		
	true urgent	true not		true normal	true not		true spam	true not		true yes	true no
system urgent	8	11	system normal	60	55	system spam	200	33	system yes	268	99
system not	8	340	system not	40	212	system not	51	83	system no	99	635

precision = $\frac{8}{8+11} = .42$

precision = $\frac{60}{60+55} = .52$

precision = $\frac{200}{200+33} = .86$

microaverage
precision = $\frac{268}{268+99} = .73$

macroaverage
precision = $\frac{.42+.52+.86}{3} = .60$

Figure 6.6 Separate contingency tables for the 3 classes from the previous figure, showing the pooled contingency table and the microaveraged and macroaveraged precision.

- **macro**: if performance is balanced across classes (calculate P, R for each class and average)
- **micro**: if nr. of cases is balanced across classes (sum all the TPs for each classification task and divide by TPs + FPs; high score for micro is based on the performance for spam)

Contingency table for sentiment

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Figure 6.4 Contingency table

Do not confuse positive and negative classes with positive and negative judgements by a system!

Contingency table

(another example for 10 tweets)

RESULTS	GOLD	SYS
tweet1	pos	pos
tweet2	pos	neg
tweet3	pos	pos
tweet4	pos	neg
tweet5	pos	pos
tweet6	pos	pos
tweet7	neg	pos
tweet8	neg	pos
tweet9	neg	neg
tweet10	neg	pos

		TRUE	
		YES	NO
PREDICTED	YES	TP	FP
	NO	FN	TN

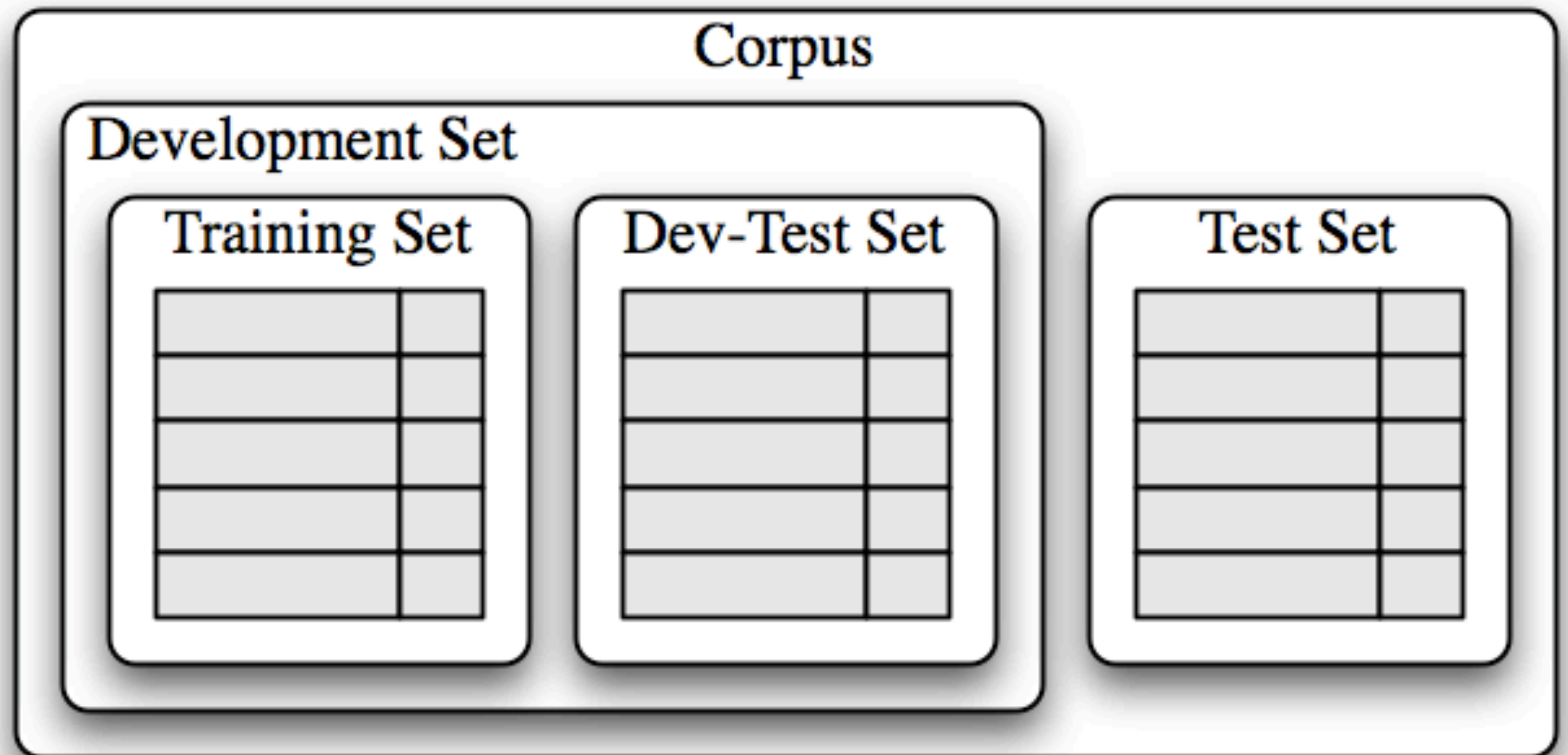
		TRUE	
		POSITIVE	NOT
PREDICTED	POSITIVE	4	3
	NOT	2	1
recall	TP/TP+FN	4/4+2	0,66
precision	TP/TP+FP	4/4+3	0,57

		TRUE	
		NEGATIVE	NOT
PREDICTED	NEGATIVE	1	2
	NOT	3	4
recall	TP/TP+FN	1/1+3	0,25
precision	TP/TP+FP	1/1+2	0,33

Create a contingency table, calculate recall and precision for each category, choose between micro or macro averaging.

Evaluation framework

Common practice to divide a dataset into 3 subsets: train, dev, test (representative of the problem)



Folded cross-validation

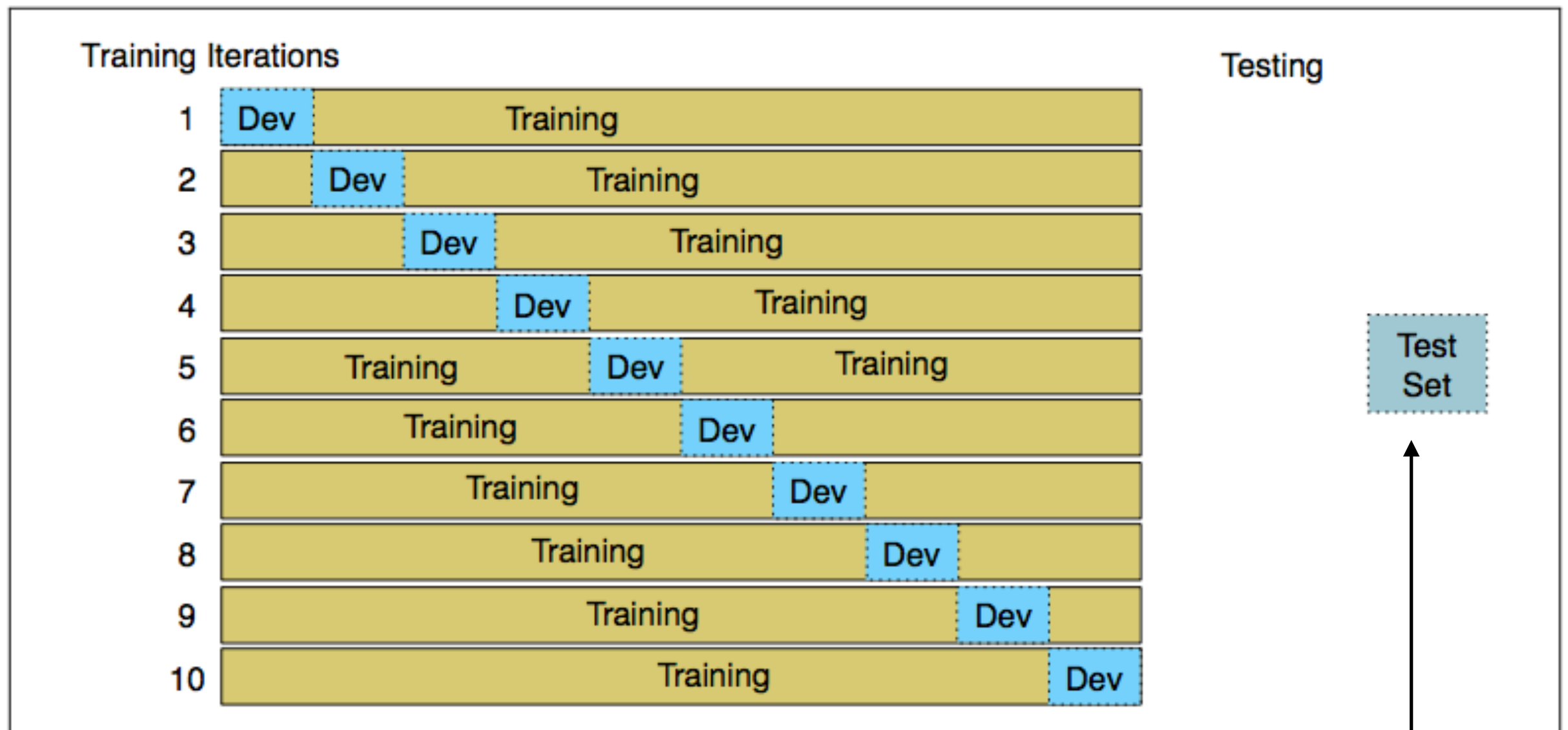


Figure 6.7 10-fold crossvalidation

Some researchers only do the cross-validation.

But this is the real test

Inference, Church 2021

Performance drops when shifting test data

Table 6: NERC CoNLL 2003 testb results.

	Precision	Recall	F1
Newsreader (ixa-pipe-nerc)	91.64	90.21	90.92
Stanford NER	-	-	88.08
Ratinov et al. (2009)	-	-	90.57
Passos et al. (2014)	-	-	90.90

← Agirre & Rigau, 2016

Reuters news wires: <https://trec.nist.gov/data/reuters/reuters.html>

Table 7: NERC Intra-document Benchmarking with Wikinews.

System	mention extent	Precision	Recall	F1
Newsreader (ixa-pipe-nerc)	Inner phrase-based	62.15	76.06	68.41
Stanford NER (all english crf distsim)	Inner phrase-based	63.53	68.21	65.79
Newsreader (ixa-pipe-nerc)	Inner token-based	72.17	79.31	75.57
Stanford NER (all english crf distsim)	Inner token-based	77.14	71.77	74.36
Newsreader (ixa-pipe-nerc)	Outer phrase-based	53.01	68.03	59.59
Stanford NER (all english crf distsim)	Outer phrase-based	52.86	59.51	55.99
Newsreader (ixa-pipe-nerc)	Outer token-based	73.40	67.20	70.16
Stanford NER (all english crf distsim)	Outer token-based	78.22	60.63	68.31

https://en.wikinews.org/wiki/Main_Page

More than 20% drop. Variation, unique properties of test not represented in the training data, annotation.

Application perspective

- **High recall and low precision:**

- High risk for missing out, low risk of acting
- No false negatives (missed alarms) but at the cost of many false positives
- e.g. tsunami alert, topic classification (provide all the relevant articles)

- **Low recall and high precision** (system acts when it is confident that the classification is correct):

- Low risk of missing out, high benefit for acting
- No false positives (wrongly acted upon) but at the cost of false negatives
- E.g. spam filter without false positives so you never need to look into your spam box and can automatically delete

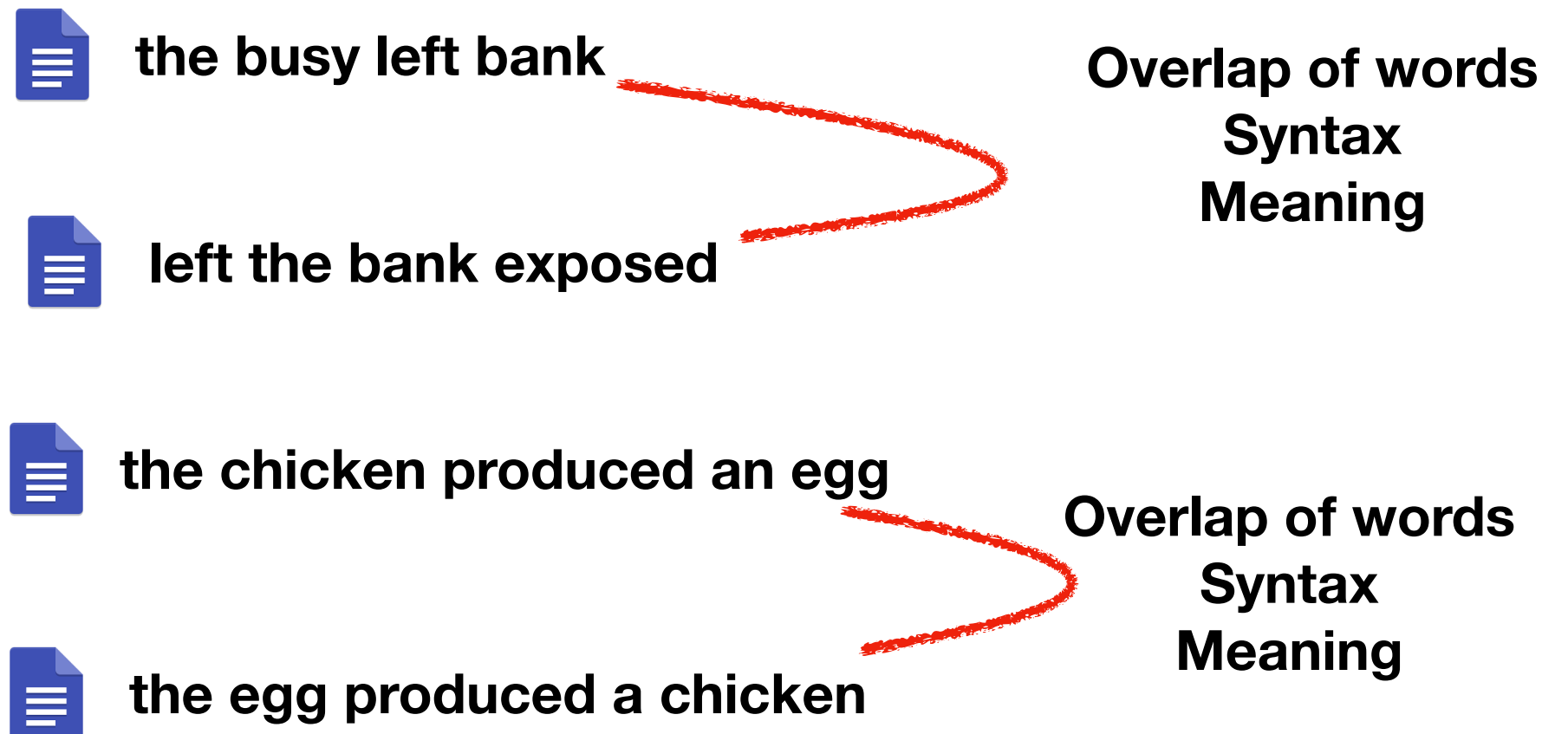
- **Common strategy:**

- first maximise the recall (solve variation and data sparseness problem) and
- next improve the precision (solve ambiguity and feature selection)

Part IV

Sequence classification

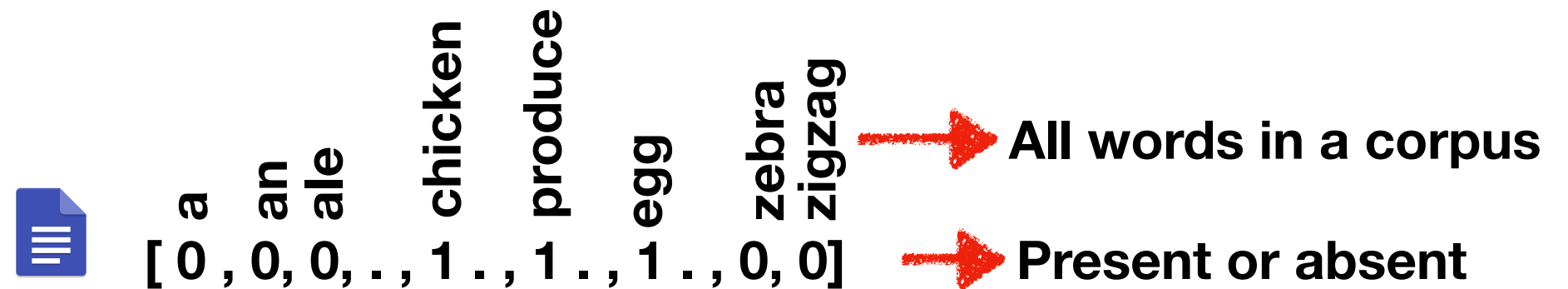
What's in a sequence?



Different meanings, similar BoW representation

What's in a sequence?

BoW VECTOR REPRESENTATIONS



the chicken produced an egg

the egg produced a chicken

Overlap of words
Syntax
Meaning

Bags of Words
leak
meaning

CoNLL IOB sequences

(properties are assigned to individual words, which are represented in a sequence)

Word	POS	Chunk	Short shape	Label
American	NNP	B-NP	Xx	B-ORG
Airlines	NNPS	I-NP	Xx	I-ORG
,	,	O	,	O
a	DT	B-NP	x	O
unit	NN	I-NP	x	O
of	IN	B-PP	x	O
AMR	NNP	B-NP	X	B-ORG
Corp.	NNP	I-NP	Xx.	I-ORG
,	,	O	,	O
immediately	RB	B-ADVP	x	O
matched	VBD	B-VP	x	O
the	DT	B-NP	x	O
move	NN	I-NP	x	O
,	,	O	,	O
spokesman	NN	B-NP	x	O
Tim	NNP	I-NP	Xx	B-PER
Wagner	NNP	I-NP	Xx	I-PER
said	VBD	B-VP	x	O
.	,	O	.	O

Sequence labeling: predict a label for each individual token given the sequence in which it occurs

We can exploit various features within the sequence, such as words and their properties

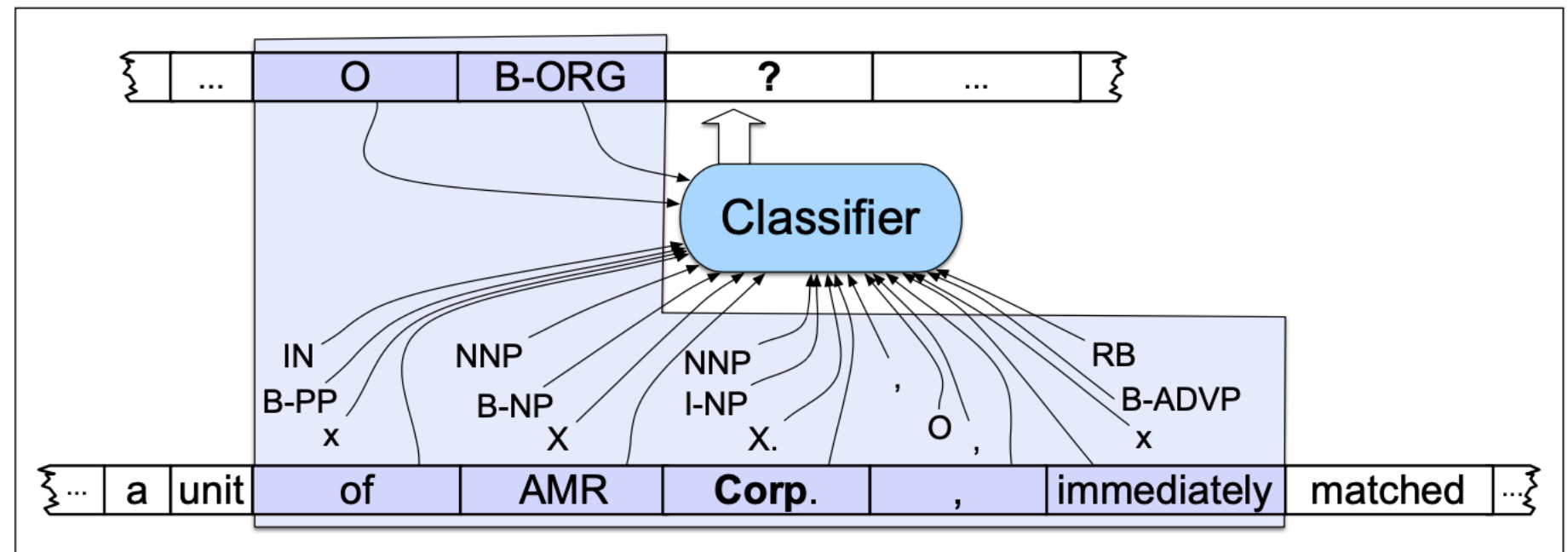
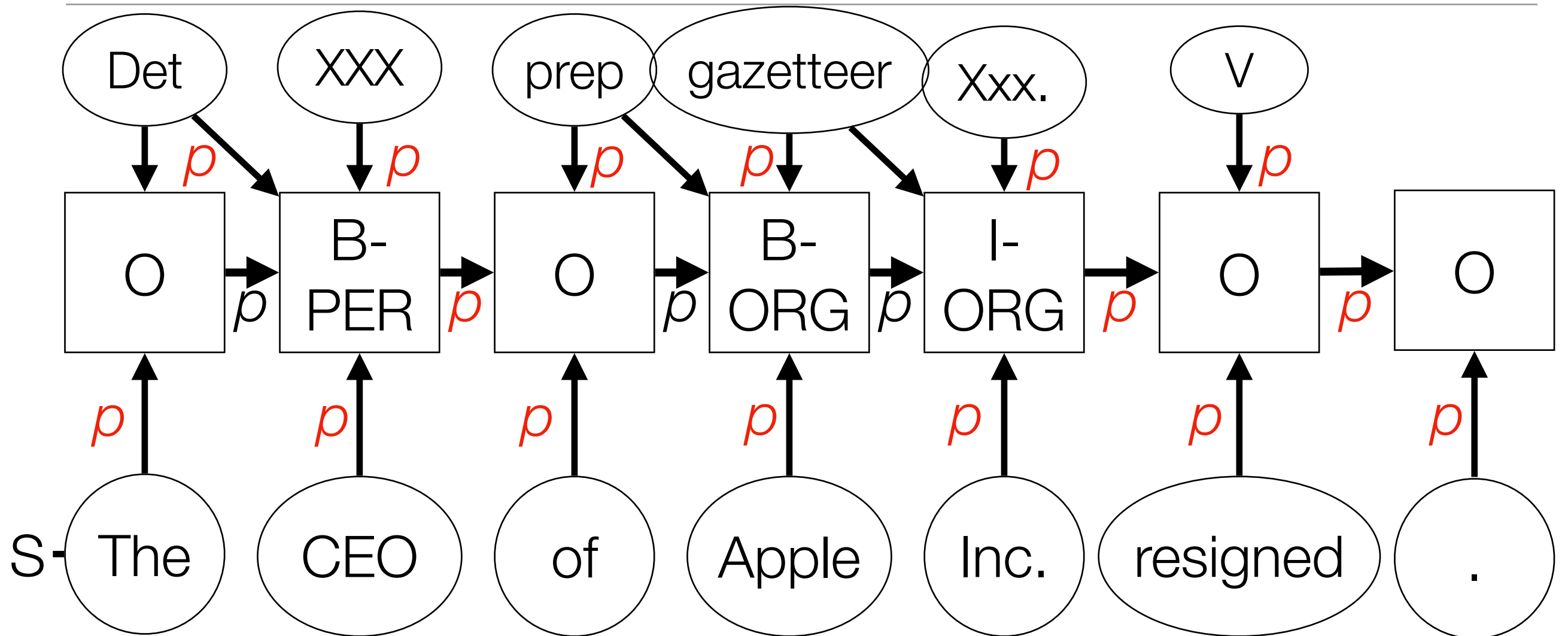


Figure 17.7 Named entity recognition as sequence labeling. The features available to the classifier during training and classification are those in the boxed area.

Sequence representations for token classification (summary)

- Instead of the complete text, each token is taken as a unit
- Represent a text (usually a sentence) as a sequence of features for each token
- Associate a label to each token in the sequence
 - Label the beginning of a named entity expression or a noun phrase
 - Represent the context as a feature of the target token
- Learn the relation between the token features and the label
- Predict the label for each token in a sequence

Conditional Random Field for IOB sequences



- Learn sequential dependencies across many features, e.g. preceding word, preceding PoS, to predict labels in addition to token properties, e.g. PoS word shape, present in gazetteers.

How to represent a sequence of tokens as a vector?

- Simple answer: a sequence of vectors, where each vector represents the features for each token
- Here is the crazy idea....
 - Each token in a sentence gets a concatenation of vectors
 - Each vector represents a feature dimension
 - lemma, PoS, dependency, position, the previous word, the next word, the previous PoS, the next PoS, etc.
 - we mark which feature value applies for a token using a one-hot-vector

A sequence of vectors for each token

- Corpus:
 - doc1: “Vaccines are safe”
 - doc2: “Vaccines cause autism”
- Vocabulary: [**vaccines, are, cause, safe, autism**]₅
- Vector to represent a document as a sequence:
 - doc1: [[1,0,0,0,0]₅ [0,1,0,0,0]₅ [0,0,0,1,0]₅]
 - doc2: [[1,0,0,0,0]₅ [0,0,1,0,0]₅ [0,0,0,0,1]₅]

A sequence of vectors for each token

- Corpus:

- doc1: “Vaccines are safe”

- doc2: “Vaccines cause autism”

- Vocabulary: [**vaccines, are, cause, save, autism**]₅

- Vector:

- doc1: [[1,0,0,0,0]₅ [0,1,0,0,0]₅ **[0,0,0,1,0]₅**]

- doc2: [[1,0,0,0,0]₅ [0,0,1,0,0]₅ [0,0,0,0,1]₅]



This is a called a one-hot-vector

- sequence of one-hot-vectors

Text sequences of different lengths

- Corpus:

- doc1: "Vaccines are safe"
- doc2: "Vaccines cause autism"
- doc3: "Vaccines do not cause autism"

- Vocabulary: [**vaccines, are, cause, save, autism, do, not**]₇

- Vector:

- doc1: [[1,0,0,0,0,0,0]₇ [0,1,0,0,0,0,0]₇ [0,0,0,1,0,0,0]₇]

- doc2: [[1,0,0,0,0,0,0]₇ [0,0,1,0,0,0,0]₇ [0,0,0,0,1,0,0]₇]

- doc3:

[[1,0,0,0,0,0,0]₇ [0,0,0,0,0,0,1]₇ [0,0,0,0,0,0,1]₇ [0,0,1,0,0,0,0]₇ [0,0,0,0,1,0,0]₇]

What is the problem here?

Text sequences of different lengths

- Corpus:

- doc1: "Vaccines are safe"
- doc2: "Vaccines cause autism"
- doc3: "Vaccines do not cause autism"

- Vocabulary: [**vaccines, are, cause, save, autism, do, not**]₇

- Vector:

- doc1: [[1,0,0,0,0,0,0]₇[0,1,0,0,0,0,0]₇[0,0,0,1,0,0,0]₇[0,0,0,0,0,0,0]₇[0,0,0,0,0,0,0]₇]
- doc2: [[1,0,0,0,0,0,0]₇[0,0,1,0,0,0,0]₇[0,0,0,0,1,0,0]₇[0,0,0,0,0,0,0]₇[0,0,0,0,0,0,0]₇]
- doc3: [[1,0,0,0,0,0,0]₇[0,0,0,0,0,1,0]₇[0,0,0,0,0,0,1]₇[0,0,1,0,0,0,0]₇[0,0,0,0,1,0,0]₇]

- Equalise the vectors:

- the tokens of the longest text with padding
- the tokens of the shortest by cutting
- the average length (padding and cutting where needed)

This is an engineering solution

padding with zeros to make them equal in length

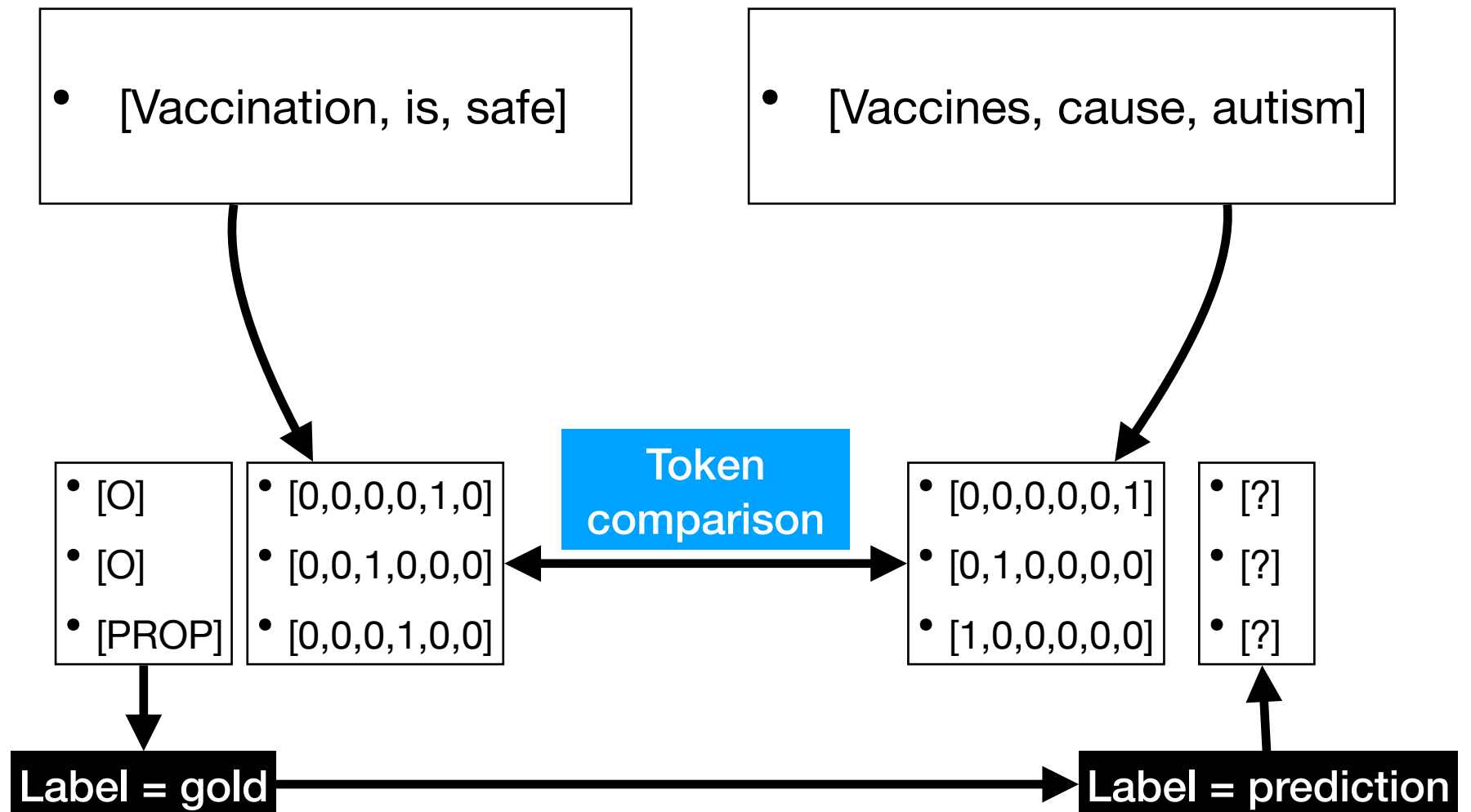
Token or Word sequences as One-Hot-Encodings

● Feature space

● vocabulary =
[autism, cause, is, safe, vaccination, vaccines]

● Vector dimensions

- [0,0,0,0,0,0]
- length = 6



- Sequence = one-hot-vectors for each token
- Associate labels with each position and we train a system to predict a label for each position in a target text
- Long and sparse vectors for each token
- Occurrence of a word is the only predictive feature
- How to add other features?

Token or Word sequences as One-Hot-Encodings

● Feature space

● vocabulary =
[autism, cause, is,
safe, vaccination,
vaccines]

● Vector dimensions

- [0,0,0,0,0,0]
- length = 6

• [Vaccines, cause, autism]

	Current Token	Previous Token	Next Token
• [O]	• [0,0,0,0,0,1]	• [0,0,0,0,0,0]	• [0,1,0,0,0,0]
• [O]	• [0,1,0,0,0,0]	• [0,0,0,0,0,1]	• [1,0,0,0,0,0]
• [PROP]	• [1,0,0,0,0,0]	• [0,1,0,0,0,0]	• [0,0,0,0,0,0]

- Previous and following words are also predictive features
- Even longer and more sparse vectors for each token
- How to add other features?

Part-of-Speech as one-hot-encoding

```
# text = Vaccines cause autism.
```

1	Vaccines	vaccine	NNP	NNP	Case=Nom Number=Plur	2	nsubj	2:nsubj
2	cause	cause	VERB	VBP	Number=Plur Person=3 Tense=Pres	0	root	0:root
5	autism	autism	NOUN	NN	Number=Plur	2	obj	2:obj
6	.	.	PUNCT	.	—	2	punct	2:punct

		NN	NNP	VBP	CC		POS-36		
Vaccines	=	[0 ,	1 ,	0 ,	0 ,	0 ,	0 ,	 ,	0]
cause	=	[0 ,	0 ,	1 ,	0 ,	0 ,	0 ,	 ,	0]
autism	=	1 ,	0 ,	0 ,	0 ,	0 ,	0 ,	 ,	0]

Syntactic dependencies as one-hot-encoding

```
# text = Vaccines cause autism.
```

1	Vaccines	vaccine	NNP	NNP	Case=Nom Number=Plur	2	nsubj	2:nsubj
2	cause	cause	VERB	VBP	Number=Plur Person=3 Tense=Pres	0	root	0:root
5	autism	autism	NOUN	NNS	Number=Plur	2	obj	2:obj
6	.	.	PUNCT	.	_	2	punct	2:punct

ID of a token (1st column)

‘Vaccines’ is the subject 1 position to the left of the word it depends on (‘cause’).

nsubj-1 **nsubj-2** **root**

obj+1 **obj+2**

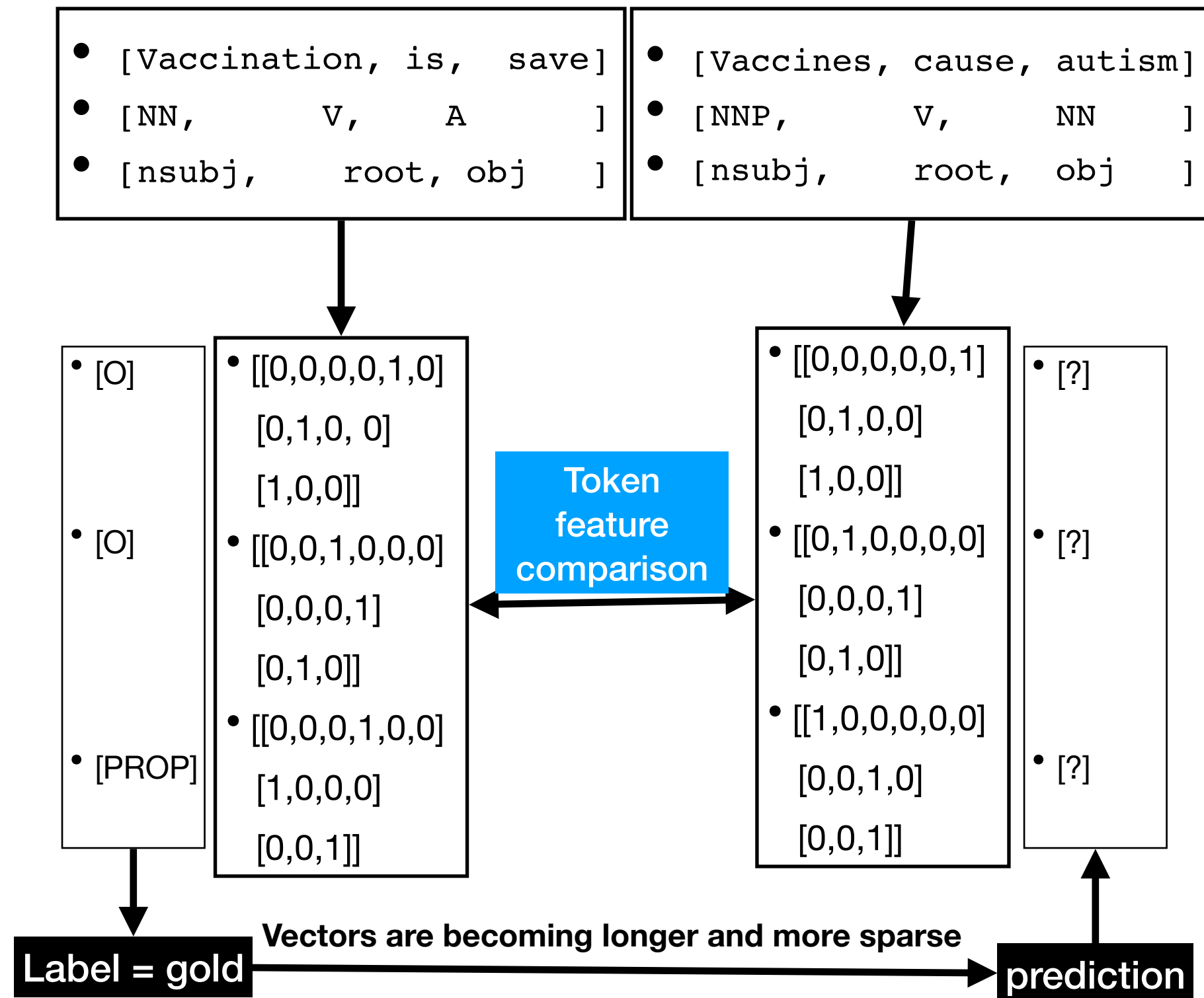
‘autism’ is the object 1 position to the right of the word it depends on (‘cause’).

Vaccines	=	[1 , 0, 0, 0, 0, 0, 0, 0,, 0]
cause	=	[0, 0, 1 , 0, 0, 0, 0, 0,, 0]
autism	=	[0, 0, 0, 0, 0, 0, 1 , 0,, 0]

Combining one-hot-encodings for sequence annotation

● Feature space

- vocabulary = [autism, cause, is, safe, vaccination, vaccines]
- PoS = [A, NN, NNP, V]
- Dependencies = [nsubj, root, obj]
- Vector dimensions
 - $[[0,0,0,0,0,0] + [0,0,0,0,0] + [0,0,0]]$



NERC data sample

<https://www.kaggle.com/abhinavwalia95/entity-annotated-corpus> !

In practice, a lot of properties -> feature ablation to identify indicative features.

- id, lemma, **next-lemma**, **next-next-lemma**, **next-next-pos**, **next-next-shape**, **next-next-word**, **next-pos**, **next-shape**, **next-word**, pos, **prev-iob**, **prev-lemma**, **prev-pos**, **prev-prev-iob**, **prev-prev-lemma**, **prev-prev-pos**, **prev-prev-shape**, **prev-prev-word**, **prev-shape**, **prev-word**, sentence_idx, shape, word, tag
- 0, thousand, of, demonstr, NNS, lowercase, demonstrators, IN, lowercase, of, NNS, __START1__, __start1__, __START1__, __START2__, __start2__, __START2__, wildcard, __START2__, wildcard, __START1__, 1, capitalized, Thousands, O
- 1, of, demonstr, have, VBP, lowercase, have, NNS, lowercase, demonstrators, IN, O, thousand, NNS, __START1__, __start1__, __START1__, wildcard, __START1__, capitalized, Thousands, 1, lowercase, of, O
- 2, demonstr, have, march, VBN, lowercase, marched, VBP, lowercase, have, NNS, O, of, IN, O, thousand, NNS, capitalized, Thousands, lowercase, of, 1, lowercase, demonstrators, O

Some nice properties of vector representations

- Very flexible and fine-grained:
 - We can translate **anything** to numeric vector values and combine these: words and sentences, word combinations, part-of-speech, word distance, dependency relations, word shape, morphology, and even images and pixels —> ***mixed modalities (!)***

$$\text{bass} = \begin{matrix} \text{fish} \\ \text{music} \\ \text{lake} \end{matrix} + \begin{matrix} \text{noun} \\ \text{verb} \\ \text{adjective} \end{matrix} + \begin{matrix} \text{subject} \\ \text{object} \\ \text{predicate} \end{matrix}$$

$\text{bass} = [1, 0, 1] + [1, 0, 0] + [1, 0, 0]$

- We can assign nuanced scalar **values** between 0 and 1 (association strength, signal strength, frequencies, include only those features that occur with a certain frequency in a dataset)
- Memory efficient (digits instead of words or strings)
- Mathematical operations: similarity, analogy, multiply, concatenate, average, weigh, subtract, etc...

Problems with sparse vectors

- Thousands of documents may have hundreds of thousands of words (tokens)
- Each token in a text is represented by the combined vectors of all features with slots for all possible values
- Vectors become extremely **large** and **sparse** (many thousands of positions, mostly empty)
- Difficult to generalise over vector representations
- Rich and complex feature engineering is needed to compensate for lack (sparseness) of data (to represent all the potential cases)

Part V

Word embeddings

Unsupervised clustering of words using context vectors

Word can be defined by its neighbours

“You shall know a word by the company it keeps” (Firth 1957, Harris 1954)

Context window of words to the left and right

- **voice**, **register** wise. The **bass range** starts at a certain **C**.
- the **alto range** is from **G3** referring to **G** below **key** of **C**
- your **family** on the **lake**, **bass fishing** is a difficult pastime
- Important information about **trout fishing** on **Lake** Taupo

[Length of the vector is the size of vocabulary used]

	C	family	fishing	G	G3	lake	key	range	register	voice
bass	[1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1]									
alto	[1, 0, 0, 1, 1, 0, 1, 1, 0, 0]									
bass	[0, 1, 1, 0, 0, 1, 0, 0, 0, 0]									
trout	[0, 0, 1, 0, 0, 1, 0, 0, 0, 0]									

Context Vectors to represent word meanings

Co-occurrence counts in a large corpus (with no supervision)
Information values



[Length of the vector is the size of vocabulary used]

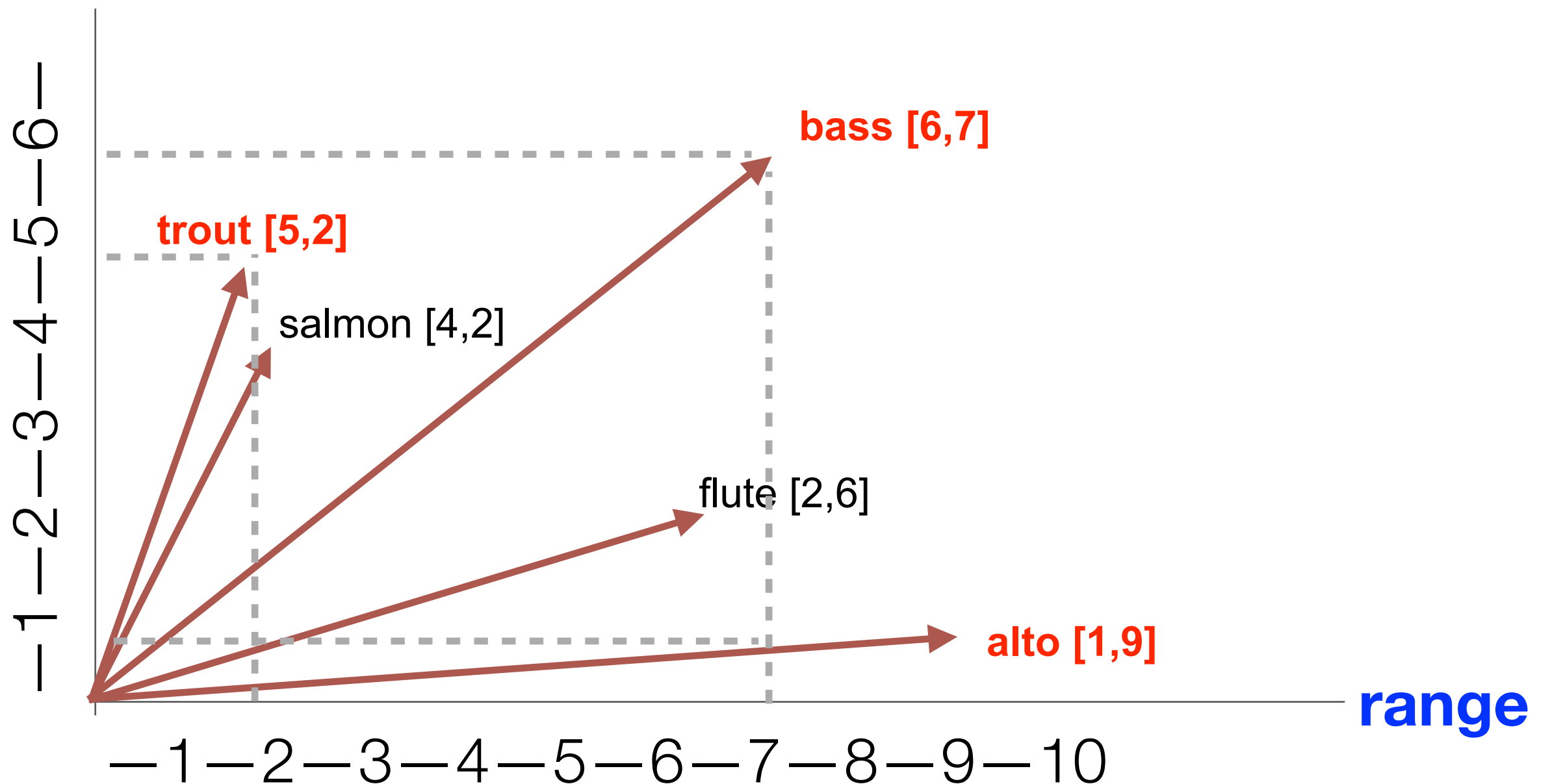
	C	family	fishing	G	G3	lake	key	range	register	voice
alto	4	0	1	1	1	0	1	9	0	0
bass	6	1	6	0	0	4	0	7	0	0
trout	0	1	5	0	0	9	0	2	0	0

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n \frac{a_i b_i}{n} = \frac{a_1 b_1 + a_2 b_2 + \cdots + a_n b_n}{n}$$

Arrows point from the terms $a_1 b_1$, $a_2 b_2$, and $a_n b_n$ in the equation to the corresponding elements in the vectors for 'alto', 'bass', and 'trout' in the table above.

Simplified view in two dimensions

fishing



Word embeddings

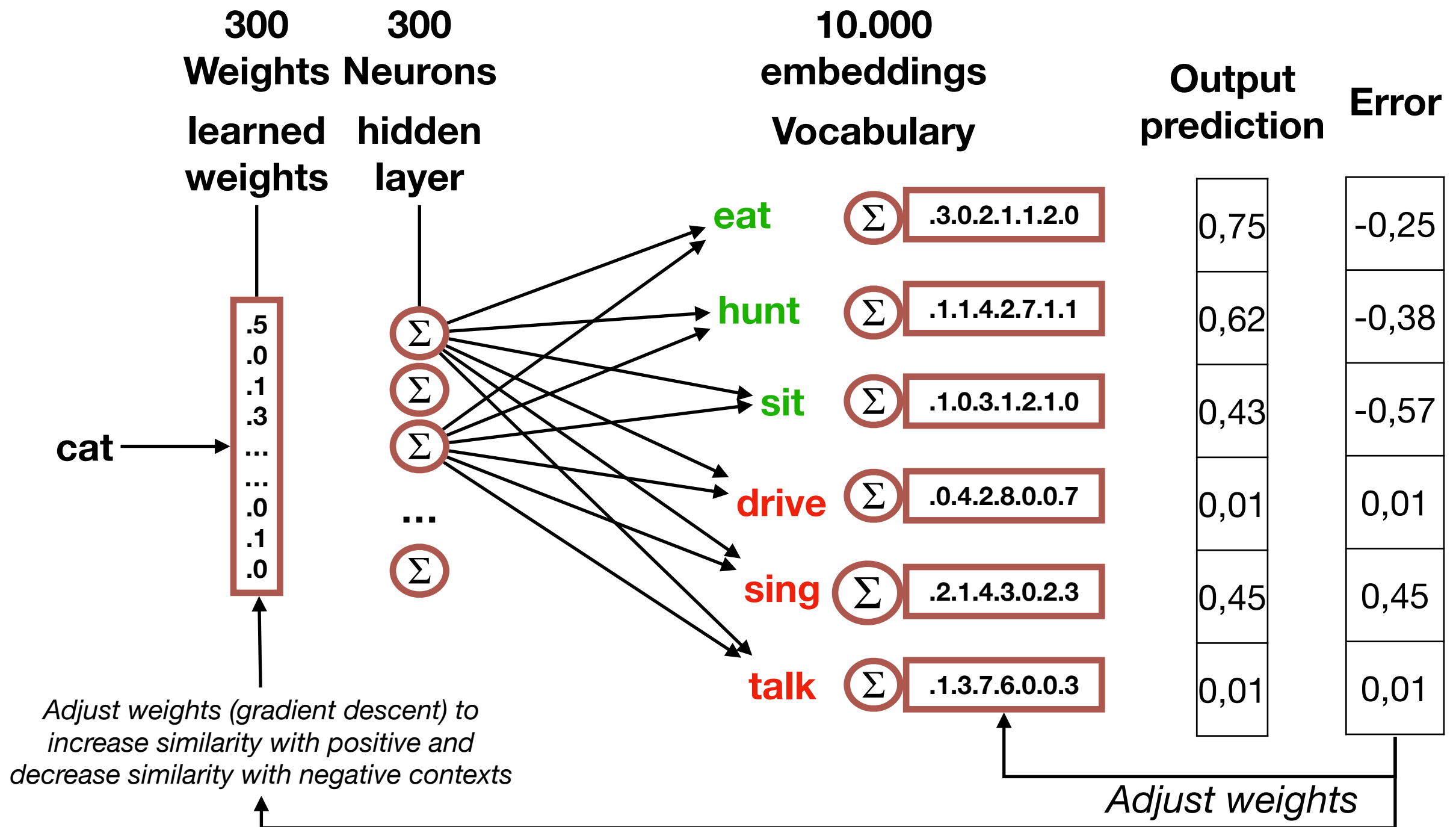
a Neural network to predict the context

Neural nets can learn to predict the context rather than represent it (word embedding is a neural net).

To learn the weights for 'cat', the system will predict the context words of 'cat'.

If it predicts 'eat' (correct), the representation of 'cat' and 'eat' become more similar (adjust the weights of 'eat' and 'cat'); 'drive' - wrong prediction (differentiate the weights of 'drive' and 'cat').

By running many times over large data, it will learn weights that are optimized for predicting context words.



Embeddings represent families of related words as features

Visualisation of embedding spaces (10K vocab, 3-dimensional space):
<http://projector.tensorflow.org> !!!

Dense, lengths of the number of nodes in the embedding layer (300-500 node)

Weights for 'cat' and 'dog' are similar: context words are similar

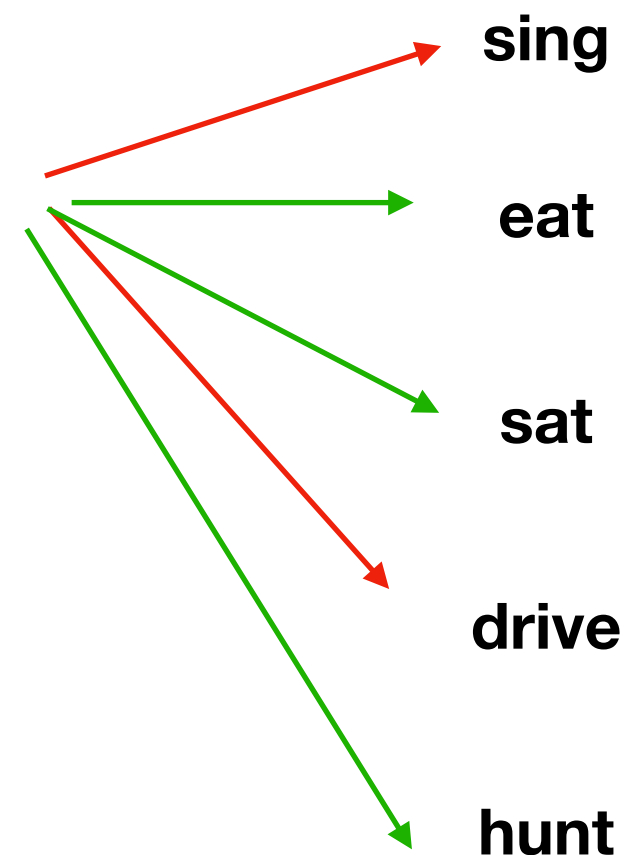
cat = [0.1, 0.4, 0.3, 0.9, 0.1, 0.8, 0.1]

dog = [0.1, 0.3, 0.4, 0.8, 0.1, 0.9, 0.1]

car = [0.2, 0.7, 0.3, 0.5, 0.1, 0.4, 0.1]

bike = [0.2, 0.7, 0.3, 0.5, 0.1, 0.4, 0.1]

Context predictions



Similarity through word-embeddings: 400 dimensions

We can now check the most similar words for any kind of words.

- gensimmodel.most_similar('vaccination')

```
[('immunisation', 0.7288434505462646),  
( 'vaccinations', 0.6694121360778809),  
( 'vaccine', 0.6497822999954224),  
( 'immunization', 0.6272329092025757),  
( 'vaccinated', 0.6225659251213074),  
( 'vaccines', 0.6096125841140747),  
( 'vaccinating', 0.5848771333694458),  
( 'immunisations', 0.5839511156082153),  
( 'immunised', 0.5639201402664185),  
( 'vaccinate', 0.562936544418335)]
```

```
vector([-0.399699, 0.06482, 0.368686, -0.377926, -0.208074, 0.092846,  
0.263108, 0.001644, -0.131597, 0.248029, 0.193516, 0.358381,  
-0.061733, 0.3458, 0.101475, -0.437568, 0.09023, -0.221989,  
-0.177281, 0.186275, -0.177728, 0.03433, 0.083448, -0.147547,  
-0.197424, 0.026185, 0.037277, 0.026462, 0.221429, 0.11221,  
-0.232993, 0.16042, -0.232213, 0.295674, 0.008174, -0.127702,  
-0.042977, 0.044021, 0.015354, -0.008699, 0.050983, 0.307894,  
-0.180218, 0.174326, -0.015698, -0.046843, 0.278651, -0.115122,  
0.105387, 0.004491, 0.094357, 0.005139, 0.13199, -0.051129,  
0.025236, 0.025753, -0.052252, -0.137729, 0.49573, 0.098202,  
0.006868, 0.110939, 0.12603, 0.296865, -0.376774, -0.060058,  
0.25929, 0.357815, 0.053625, -0.095404, -0.080501, 0.233063,  
0.093019, -0.043595, 0.472759, -0.173012, 0.001466, 0.077951,  
0.222487, 0.019556, 0.237507, -0.160088, 0.117915, 0.179803,  
0.107175, 0.142339, 0.046431, 0.423276, -0.157801, 0.221353,
```

- gensimmodel.most_similar('autism')

```
[('autistic', 0.709283709526062),  
( 'adhd', 0.6003193855285645),  
( 'dyslexia', 0.5945056676864624),  
( 'dyspraxia', 0.5788421034812927),  
( 'schizophrenia', 0.5415143966674805),  
( 'high-functioning', 0.5388866662979126),  
( 'aspergers', 0.5324583053588867),  
( 'me/cfs', 0.5201038718223572),  
( 'asd', 0.5089726448059082),  
( 'neurodevelopmental', 0.5016077160835266)]
```

```
vector([ 2.97010e-02, 1.77924e-01, -9.36180e-02, -7.38020e-02,  
-2.63494e-01, 5.39020e-02, 1.31954e-01, 1.20233e-01,  
-1.05351e-01, 1.14572e-01, 3.09344e-01, 6.28220e-02,  
2.01120e-02, -9.78420e-02, 1.20324e-01, 2.33840e-02,  
1.49410e-01, -3.08743e-01, -1.76643e-01, -4.06421e-01,  
-6.07290e-02, -2.45634e-01, 2.38010e-01, 1.11927e-01,  
-4.30489e-01, -4.17800e-02, 8.06020e-02, 1.17752e-01,  
-2.21880e-02, -9.04810e-02, -1.72199e-01, 1.33377e-01,  
5.44661e-01, 1.62586e-01, -4.16860e-02, -2.54711e-01,  
2.82401e-01, -8.09320e-02, -1.66021e-01, 1.35894e-01,  
-2.60487e-01, 1.47950e-01, 2.57920e-02, -1.48656e-01,  
-2.07999e-01, -2.04788e-01, 1.16318e-01, 2.79210e-01,  
-1.65796e-01, -4.18540e-02, 1.07020e-02, 6.09600e-03,  
-2.67482e-01, -2.57364e-01, 8.29890e-02, -4.44768e-01,  
2.52700e-02, 9.05790e-02, 2.24890e-01, 3.46180e-01,  
1.56335e-01, 3.06857e-01, 1.90560e-01, 9.49000e-03,
```

King - Man + Woman = ??

Problems for context vectors

- Similarity or relatedness: round, yellow, fruit, celestial object (related in what aspect?)

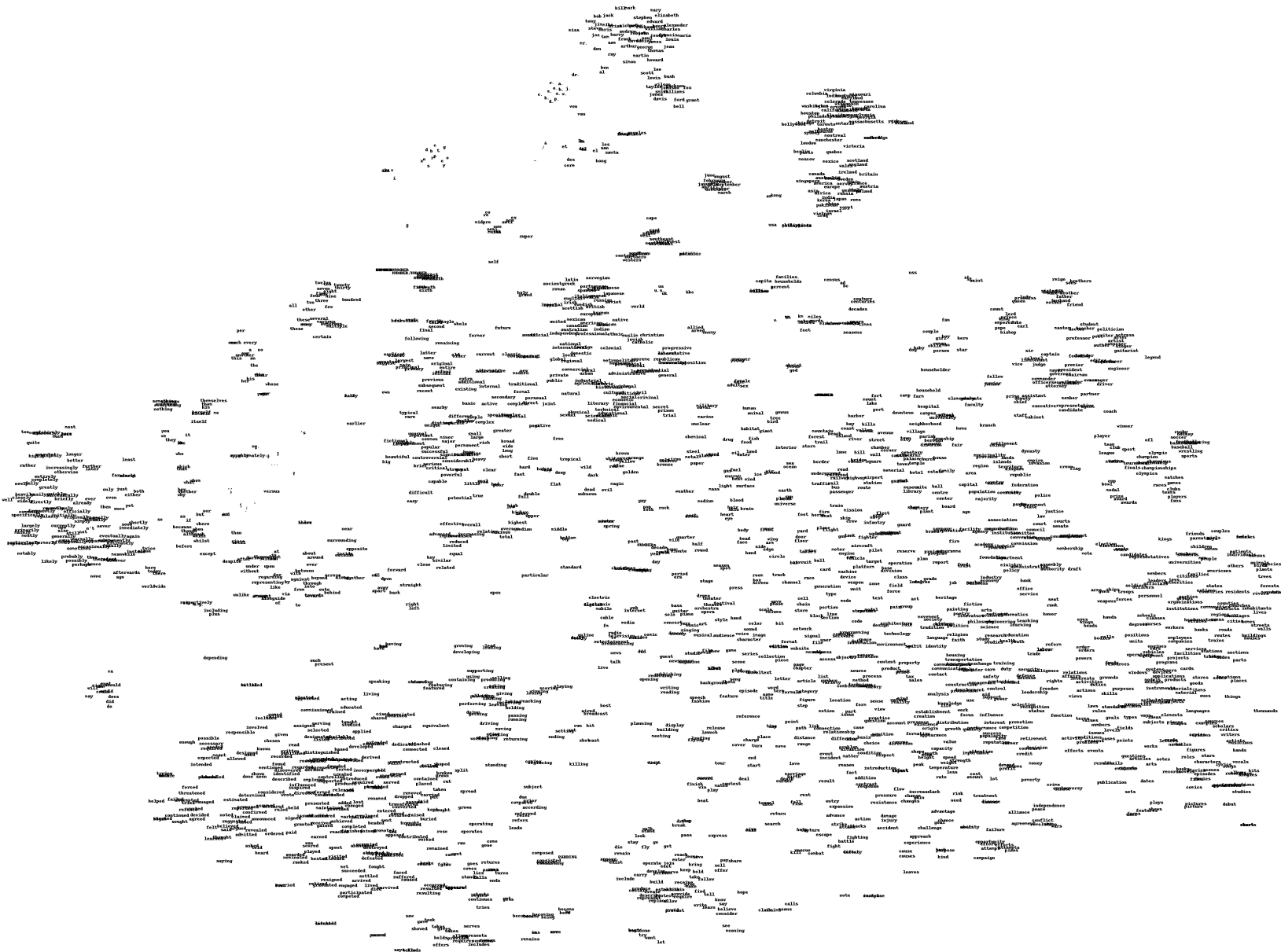


- Polysemy (words can have different meanings): star, bass, bat (embeddings don't differentiate this)

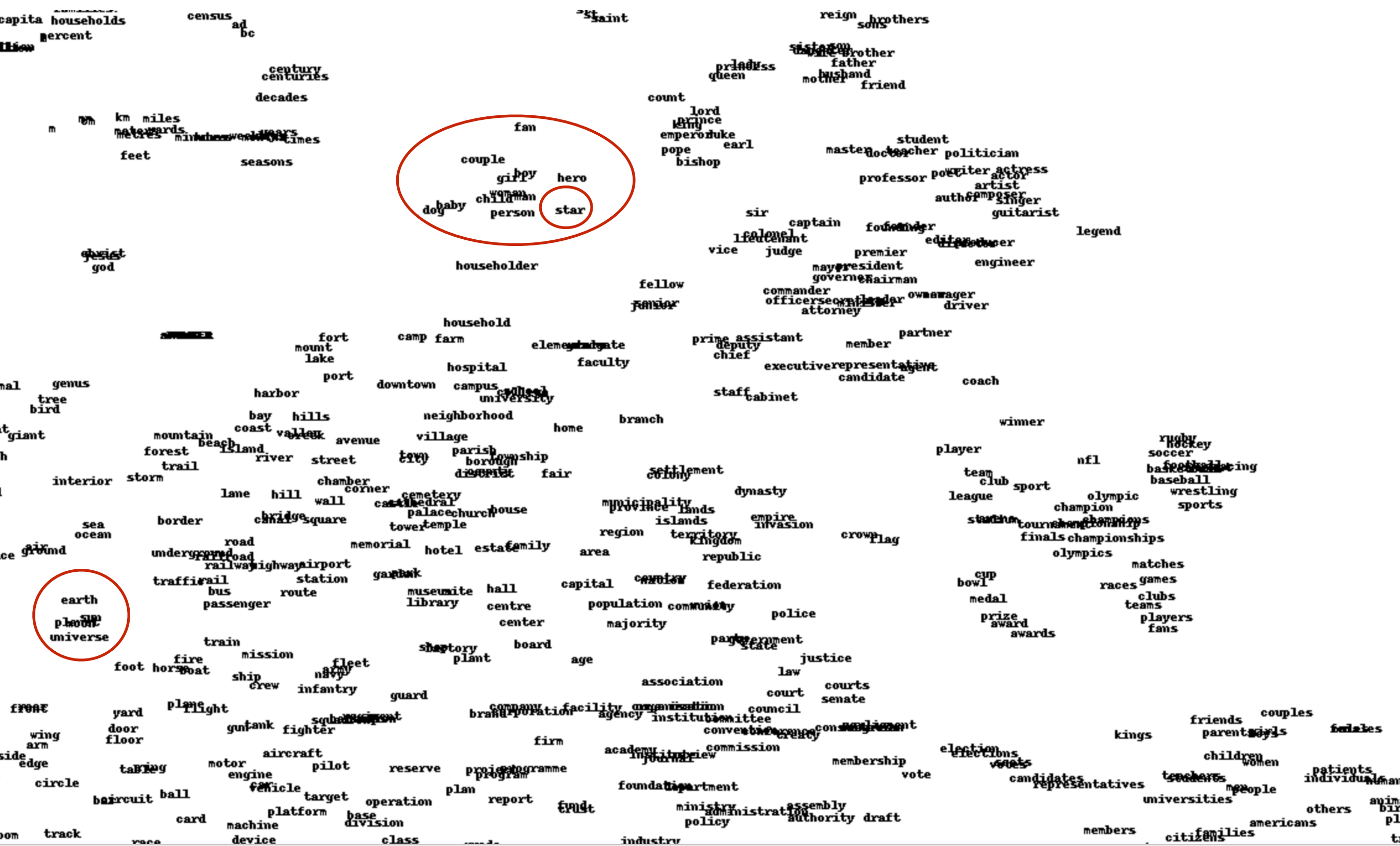


Universe of 2,500 words

(example of a space)

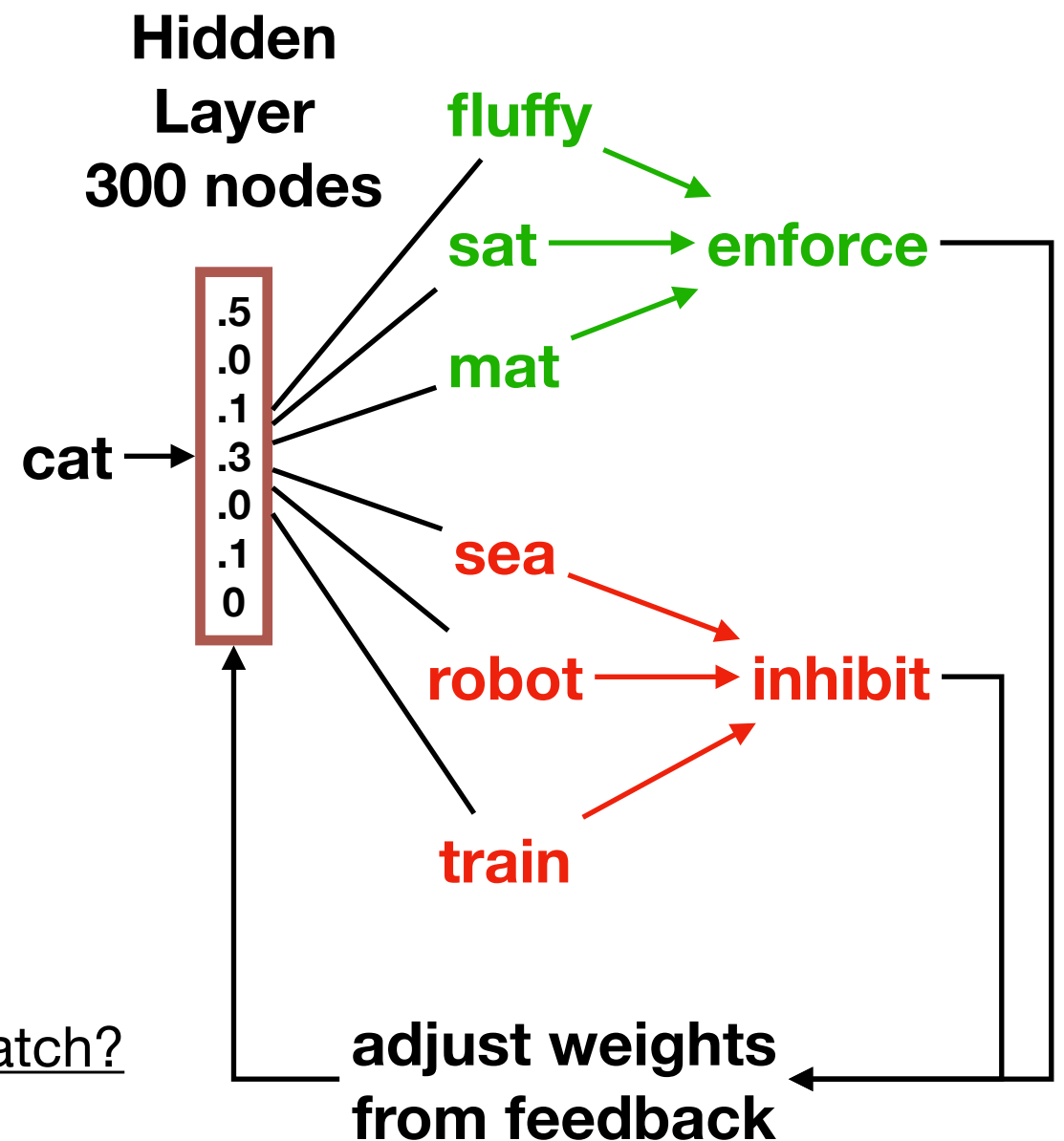


Joseph Turian's map of 2500 English words produced by using t-SNE on the word feature vectors learned by Collobert & Weston, ICML 2008



Word2Vec (Mikolov et al., 2013)

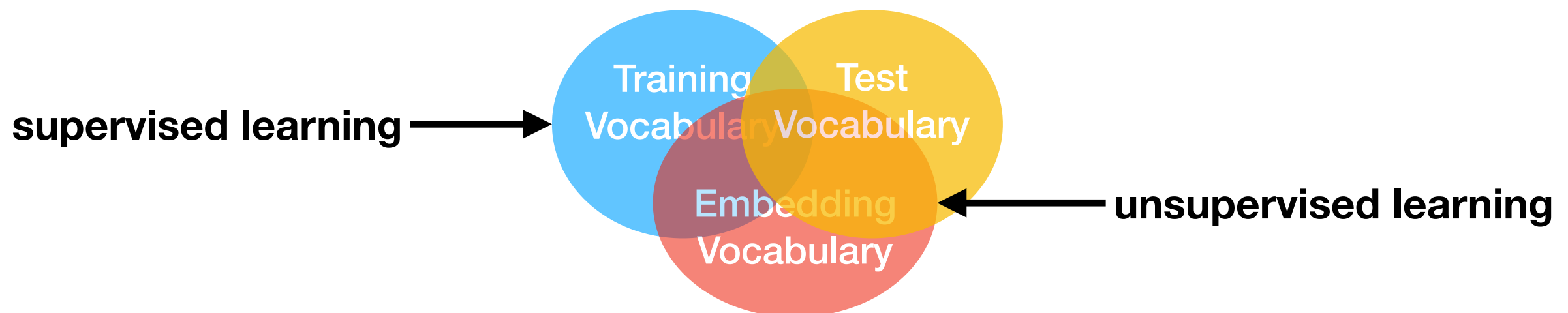
- Break down contexts of a word into pairs (positive examples, e.g., cat - sat)
 - Create random pairs as negative examples, e.g., cat - sea
 - Other tricks, e.g., focusing on words with a particular frequency, considering words in a close or wide neighborhood → different representations.
- The fluffy cat sat on the mat
 - The **fluffy** **cat**
 - **fluffy** **cat** **sat**
 - **cat** **sat** on the **mat**
 - **sat** on the **mat**



Word2Vec toy example: <https://www.youtube.com/watch?v=f7o8aDNxf7k&list=LL&index=21>

How to use embeddings for text classification

- Replace words in training and test data by their embeddings
- Short, dense and nuanced vectors instead of long, sparse one-hot-vectors
- Better generalisation for semantically related words
- Match words in test set even if not in training data (as long as they are represented in the word embedding vocabulary)



Bag-of-embeddings

- Corpus:
 - doc1: "Vaccines are safe"
 - doc2: "Vaccines cause autism"
- Word embeddings with four dimensions.

- vaccines = [.23 .45 .21 .78]

- autism = [.12 .34 .11 .32]

- are = [.01 .41 .05 .91]

- cause = [.02 .32 .08 .83]

- safe = [.22 .51 .93 .31]

- **Average** document embedding:

- doc1: [.145, .4275, .325, .58]

- doc2: [.1475, .405 .3325 .56]

Average vector representation

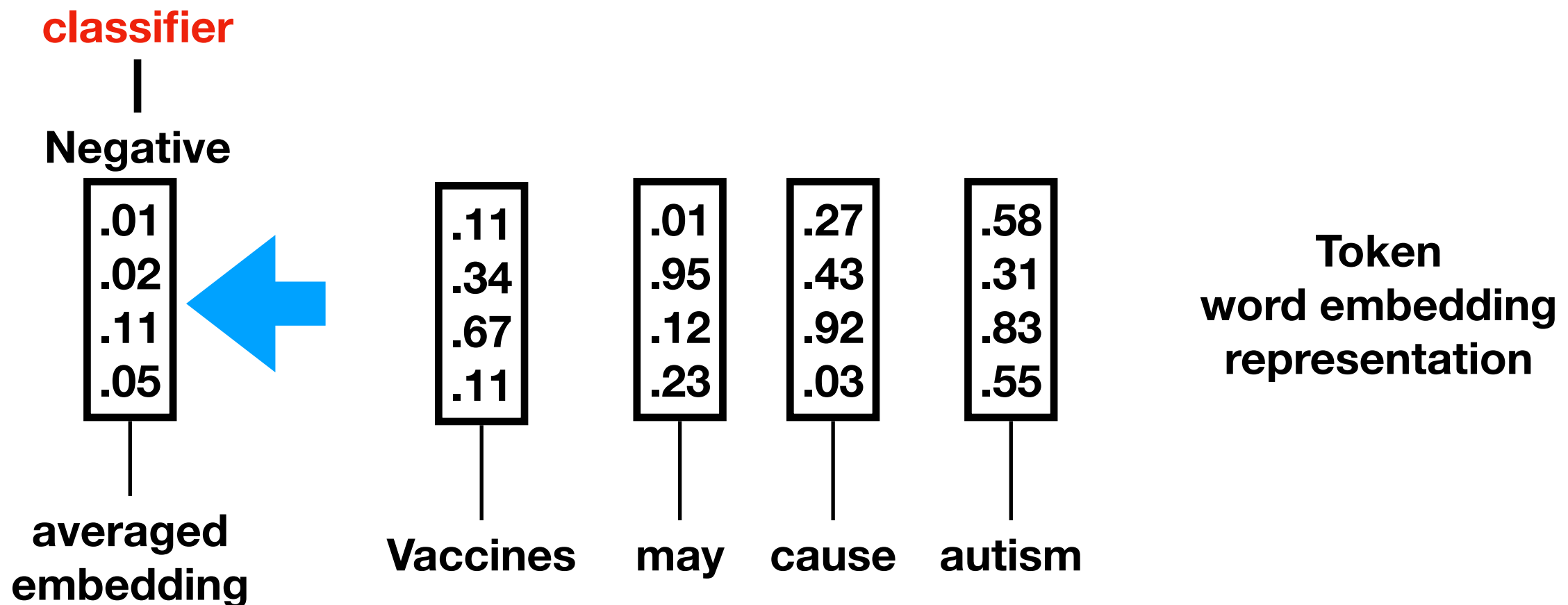
```
[1]: vectors = [[0.23,0.45, 0.21, 0.78],  
               [0.12,0.34, 0.11, 0.32],  
               [0.01,0.41, 0.05, 0.91],  
               [0.22,0.51, 0.93, 0.31]]
```

```
[2]: sum_vec = [0.0, 0.0, 0.0, 0.0]  
for vec in vectors:  
    for i, v in enumerate(vec):  
        sum_vec[i] += v  
print('Sum of the vectors:', sum_vec)  
  
average_vec = [0, 0, 0, 0]  
for i, v in enumerate(sum_vec):  
    average_vec[i] = sum_vec[i]/len(vectors)  
  
print('Average of the vectors:', average_vec)
```

```
Sum of the vectors: [0.58, 1.71, 1.3, 2.3200000000000003]  
Average of the vectors: [0.145, 0.4275, 0.325, 0.5800000000000001]
```

Training a bag-of-embeddings

- Use averaged embedding to represent a full text as a single vector (alternative approach is doc2vec)
- Some info is lost in averaging, but each word contributes to each dimension
- Train a classifier (e.g. NB, SVM) to predict labels for the averaged embeddings



Sequence representation with one-hot-encodings & word embeddings

● Feature space

● vocabulary = [**word-embedding**]

● PoS = [A, NN, NNP, V]

● Dependencies = [nsubj, root, obj]

● Vector dimensions

- [[300 embedding] + [0,0,0,0,0] + [0,0,0]]

● [Vaccination, is, save]
● [NN, V, A]
● [nsubj, root, obj]

● [Vaccines, cause, autism]
● [NNP, V, NN]
● [nsubj, root, obj]

● [[.2,.1,.2,.5,1,.6,.1]
[0,1,0, 0]
[1,0,0]]
● [[.8,.1,.2,.5,1,.6,.1]
[0,0,0,1]
[0,1,0]]
● [[.1,.1,.1,.3,1,.2,.5]
[1,0,0,0]
[0,0,1]]

Token comparison

● [[.1,.1,.1,.3,1,.2,.4]
[0,1,0,0]
[1,0,0]]
● [[.3,.1,.2,.1,1,.3,.2]
[0,0,0,1]
[0,1,0]]
● [[.2,.1,.2,.5,1,.6,.1]
[0,0,1,0]
[0,0,1]]

● most similar to [.2,.1,.2,.5,1,.6,.1]

● medicine(s), antibiotics, penicillin, medication, insulin, treatment, injection, prescription, drug(s), therapy

Units of vector representations

(summary)

- Bag of words (tokens) for full documents with label:

- doc1: [0,1, 0, 1, 0, 1, 0, 0, 1]

- doc2: [1,0, 0, 1, 0, 0, 1, 0, 1]

Sparse

**Nr. Dimensions
== vocabulary length**

- Averaged embeddings with a label:

- doc1: [.34, .76, .31, .07, .12]

- doc2: [.12, .98, .23, .03, .43]

Dense

**Nr. Dimensions
== embedding size**

- Sequence of tokens representation with label:

- word_t, PoS, dependency, word_{t-1}, word_{t+1}:

- doc1:[0,1,0,0,0,0,0,0,..., 0]_{t1},..., [0,1,0,0,0,0,0,0,..., 0]_{tn}

- doc2:[0,0,0,0,1,0,0,0,..., 0]_{t1},..., [0,0,0,0,1,0,0,0,..., 0]_{tn}

**Sparse
one-hot-encoding**

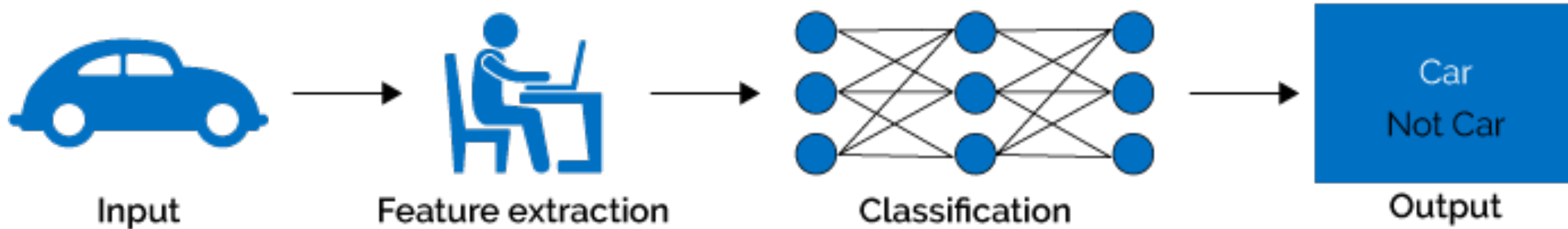
**Nr. Dimensions
== vocabulary length
+ PoS values
+ dependency labels
+ vocabulary length
+ vocabulary length**

Part VI

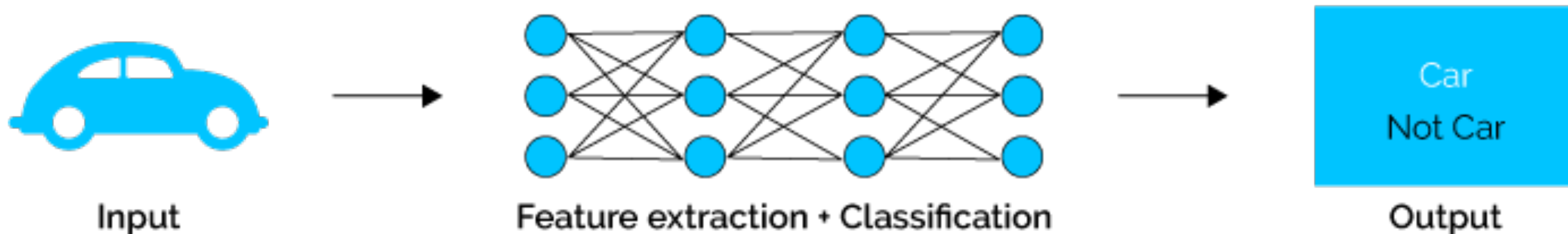
Contextual embeddings

Deep learning

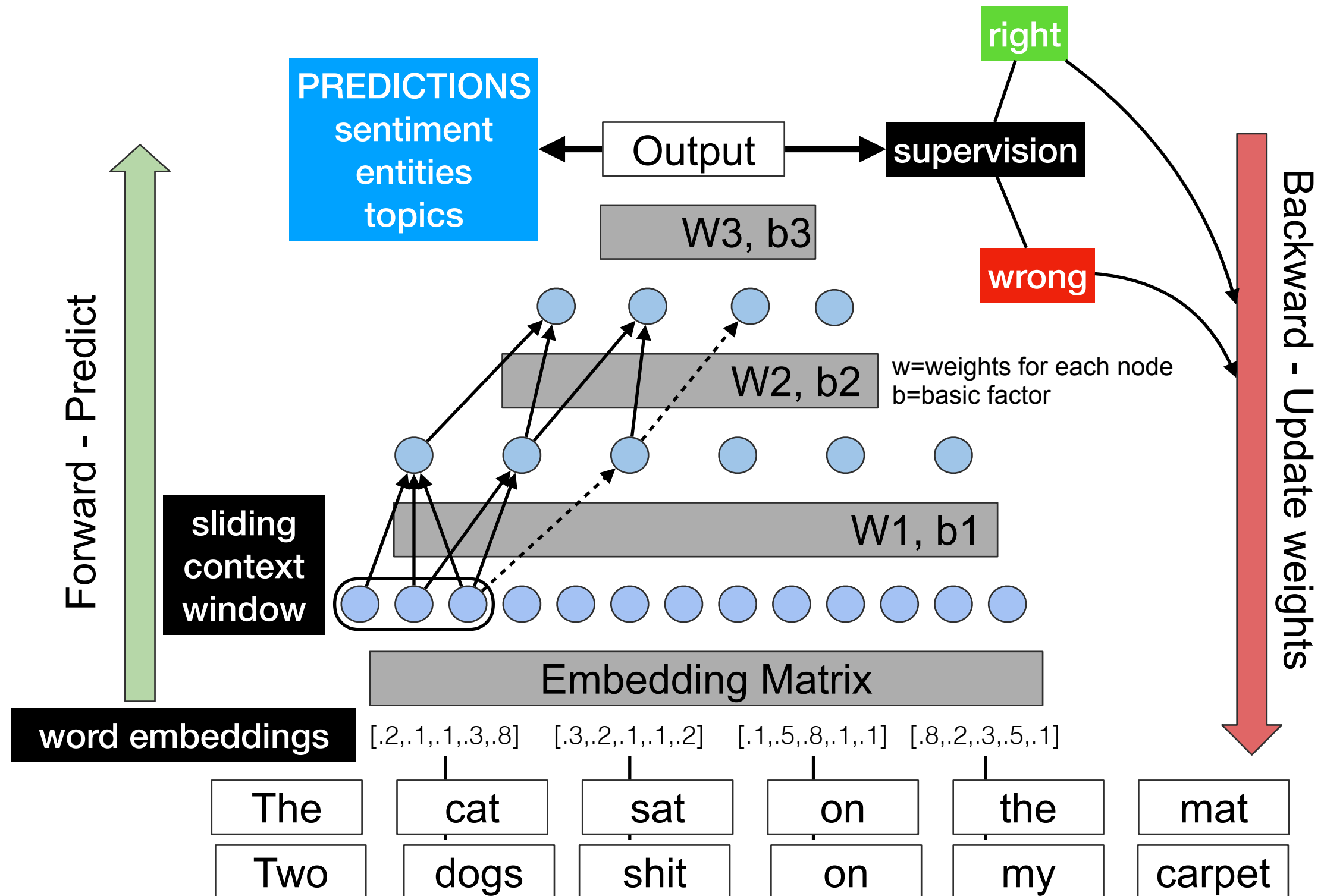
Machine Learning



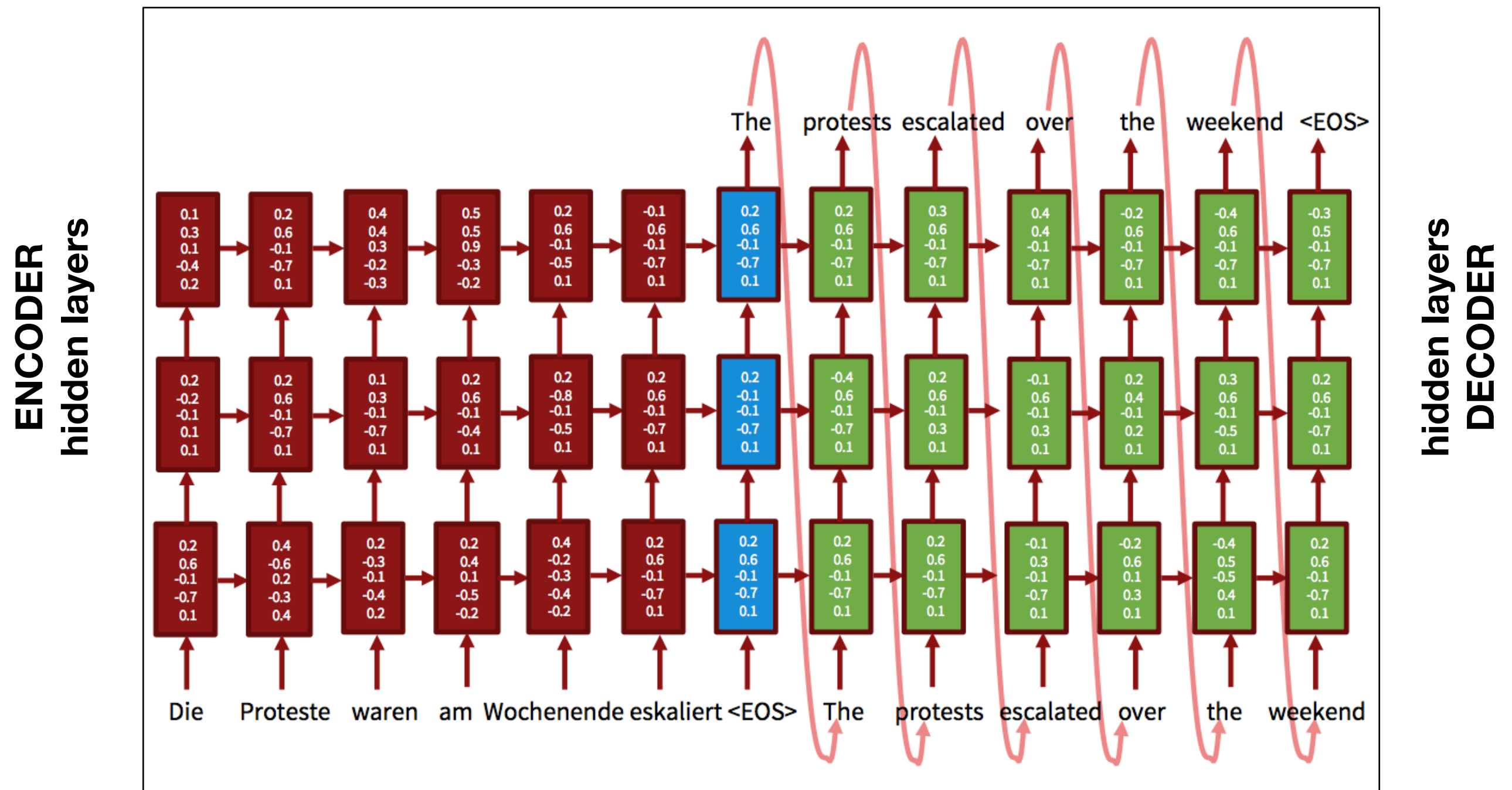
Deep Learning



Convolutional neural network



Long-Short-Term-Memory (LSTM)



Google Neural MT

Transformer models: self-attention

- Each token embedding can be modified by any other token embedding in their context (e.g., sentence)
- During training the model learns which tokens from the context are important for a target token because it helps performing a task: predicting *masked* context words
- The position in the sequence is also encoded in a layer
- Different layers of encoding blocks are stacked on each other

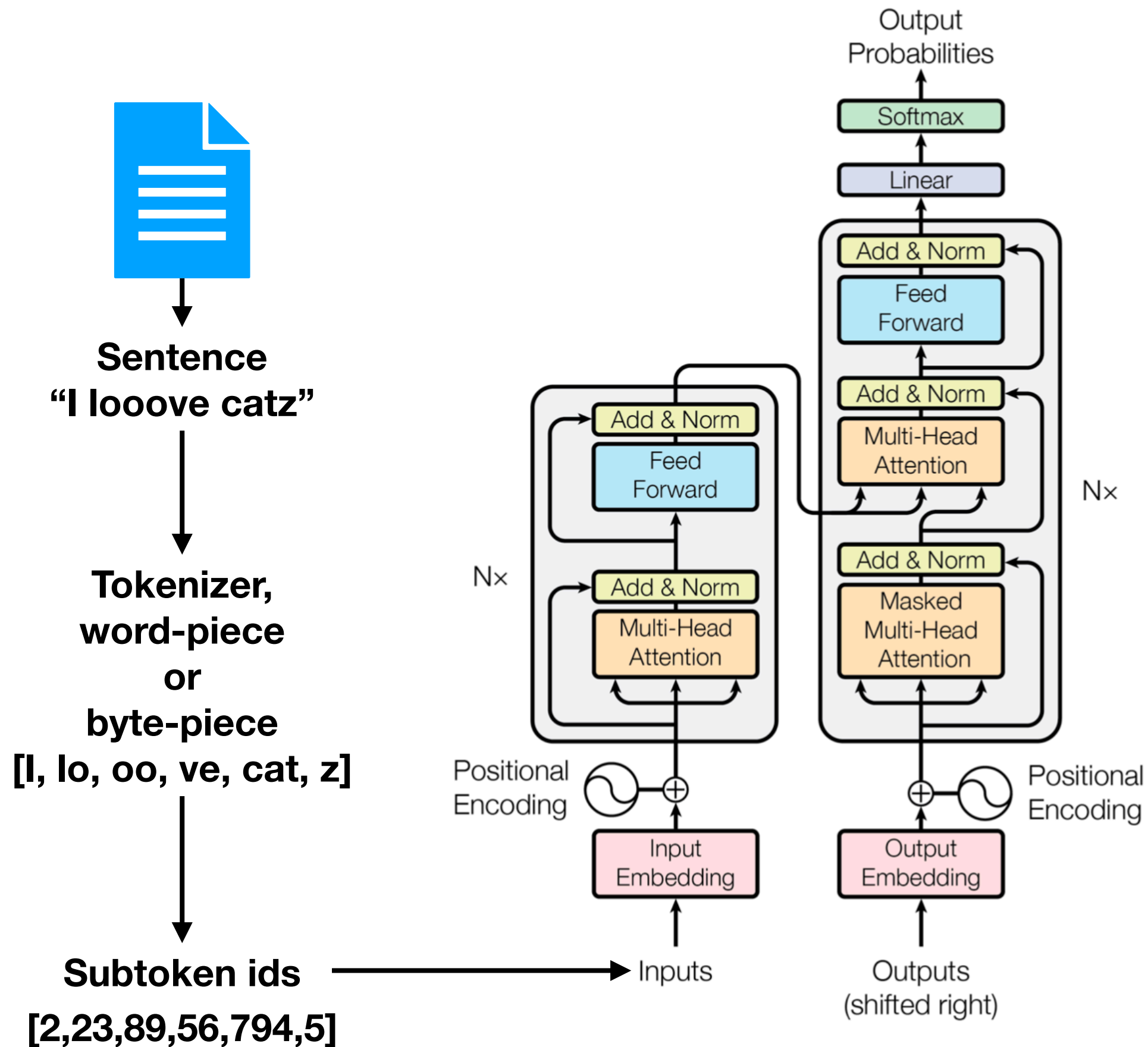


Figure 1: The Transformer - model architecture.

Vocabulary

multilingual BERT

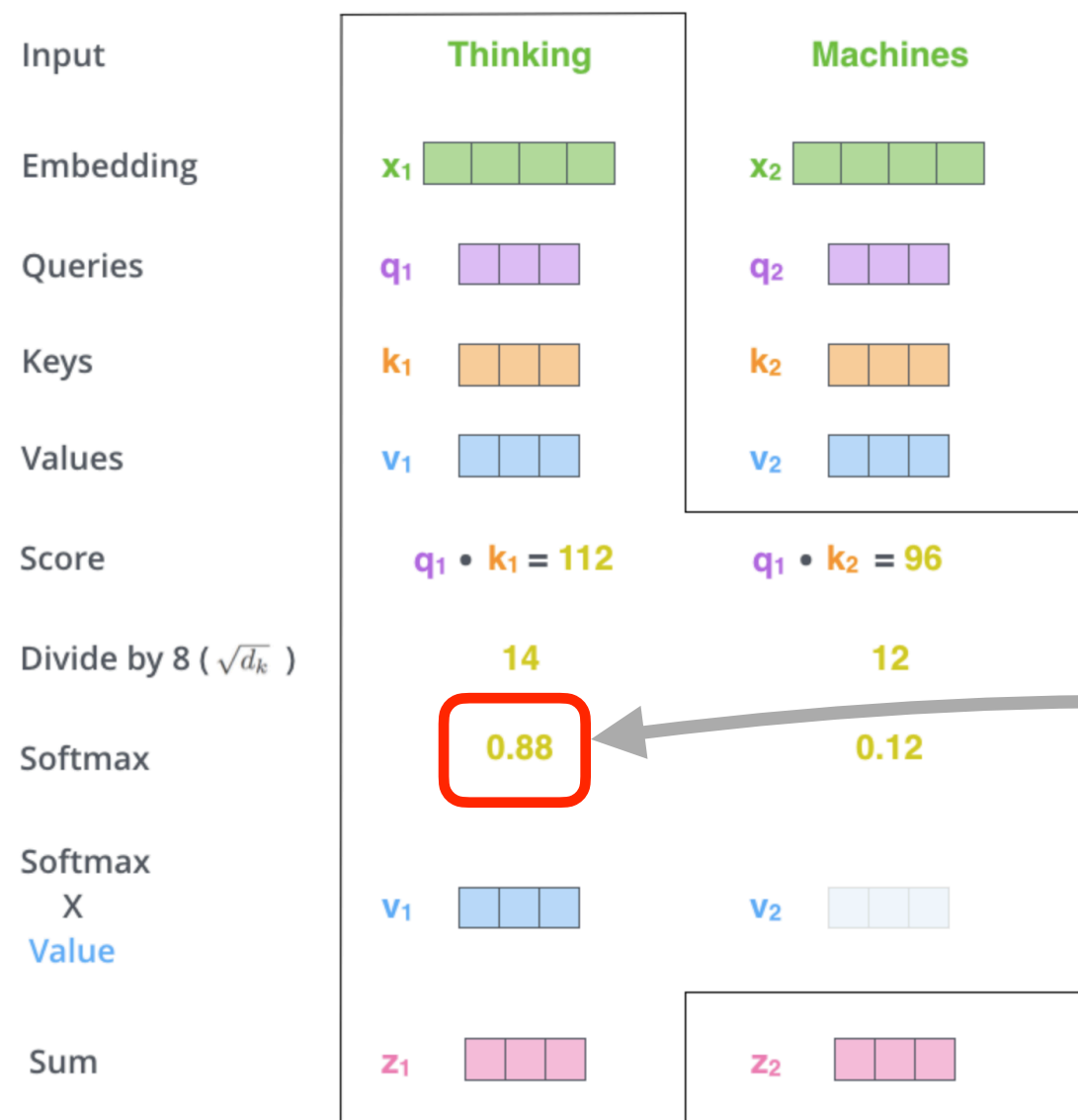
鮫, 鰈, 峯, 颞, [?], [?], 哧, 缦, 蘋, 鮑, de, the, in, sa, la, ##s, of, en, and, ##e, ##a, ang, to, ##n, ##i, el,
 der, un, di, die, nga, ##t, на, on, que, is, se, den, le, ##r, ##o, ar, del, na, ##u, et, und, for, was, da,
 ##en, des, an, il, as, van, 10, je, ##m, by, les, es, do, una, nasa, 2011, ##y, ##l, at, al, er, ##d, og, ##es,
 km, ##k, von, du, 11, with, 2000, con, per, los, °c, ##er, ##и, av, med, no, est, 2010, het, ##a, och,
 2012, dem, 12, por, he, su, si, som, from, 2014, it, 2015, 2007, 20, species, pa, that, earth, im, ##na,
 een, ##an, ja, han, ##ta, 2016, ..., dan, 2009, das, 15, ##te, ni, ##ne, ug, va, isbn, 2008, 30, ##e, 24,
 world, 2013, om, 2006, 18, als, mit, his, ##y, ka, te, para, ha, com, за, ##h, في, como, new, ##re, data,
 une, ##da, ##g, em, os, observations, 14, 16, au, ##us, 21, ##м, ##c, ##in, par, 13, las, در, panoramas,
 viewfinder, palibot, gikan, sie, dans, ##on, det, ##ni, 2005, ##ch, ##ia, nasod, 25, ##ra, ##de, che, 19,
 ##ing, je, life, 17, son, yang, по, 000, az, 2017, ##ja, 2004, so, geon
 op, ce, od, nel, 2002, ##is, ##z, ve, war, ##ی, milimetro, um, ne, wurc
 database, ##os, ##ы, ##no, ##ti, zu, من, september, var, ##la, 23, #
 were, pour, sur, am, be, har, co, ##i, auf, ##do, ##se, ##o, za, org,
 1999, به, ii, ##to, all, ##nt, года, ##ar, bulan, this, ##p, mga, ##va, #
 ##l, και, kasarangang, ##т, lo, film, mm, ##ed, ou, ##ом, kilometro,
 ##я, ##ca, first, post, entre, finns, had, ##li, sich, ny, ##06, ۛ, bahir
 ##ce, ##ng, nam, ed, ##ri, dos, ان, ##j, ur, eta, may, во, roku, ##v, u
 trakten, history, enero, dari, rainfall, 100, 1997, ulan, да, ch

Tokenizers	
Name	Ex. Uses
Character-Level BPE	NMT, GPT
Byte-Level BPE	GPT-2 +
WordPiece	BERT
SentencePiece	XLNet
Unigram	LM
Character	Reformer
Custom	Bio-Chem

Self-attention

- <http://jalammar.github.io/illustrated-transformer/>

*a solar system has a **star** and planets*
*a movie needs a **star***



$$q_1 = x_1 \cdot W^Q$$

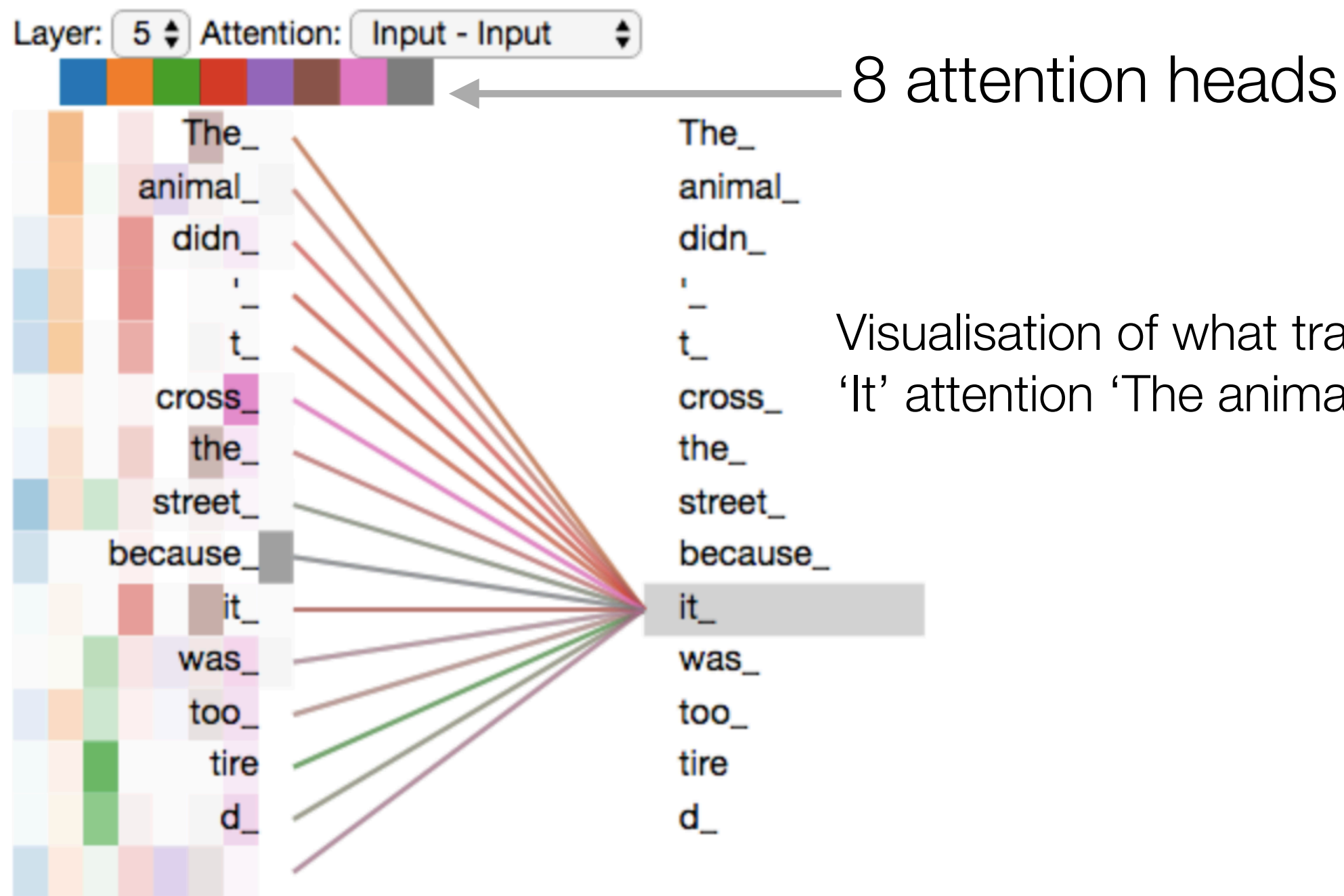
$$k_1 = x_1 \cdot W^K$$

$$v_1 = x_1 \cdot W^V$$

Most attention goes to the token itself but some also goes to other tokens in the context

Self-attention

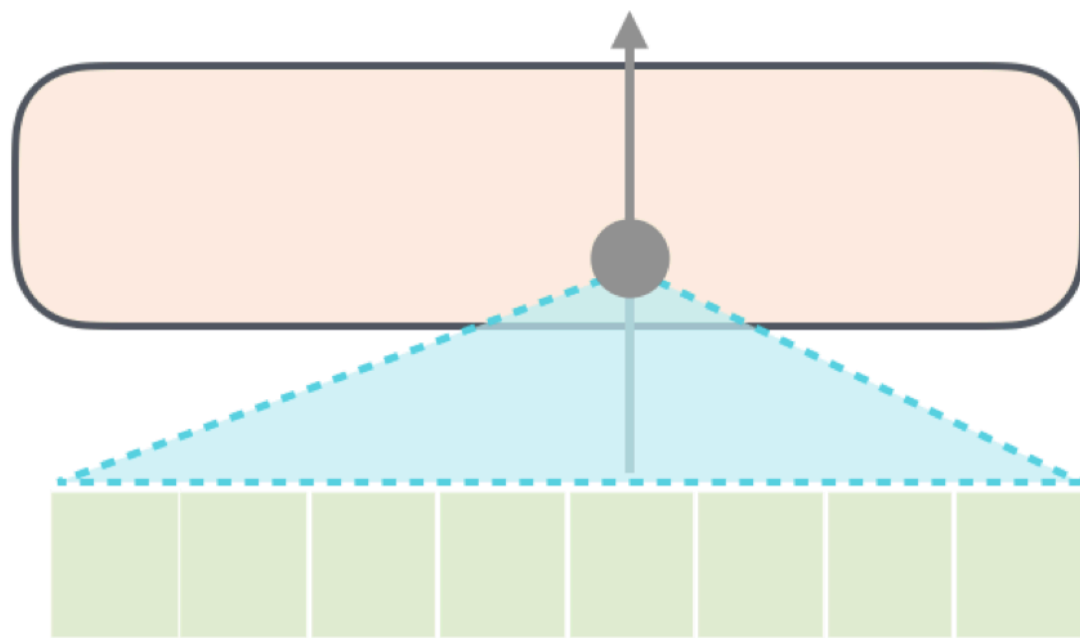
- <http://jalammar.github.io/illustrated-transformer/>



Two types of attention

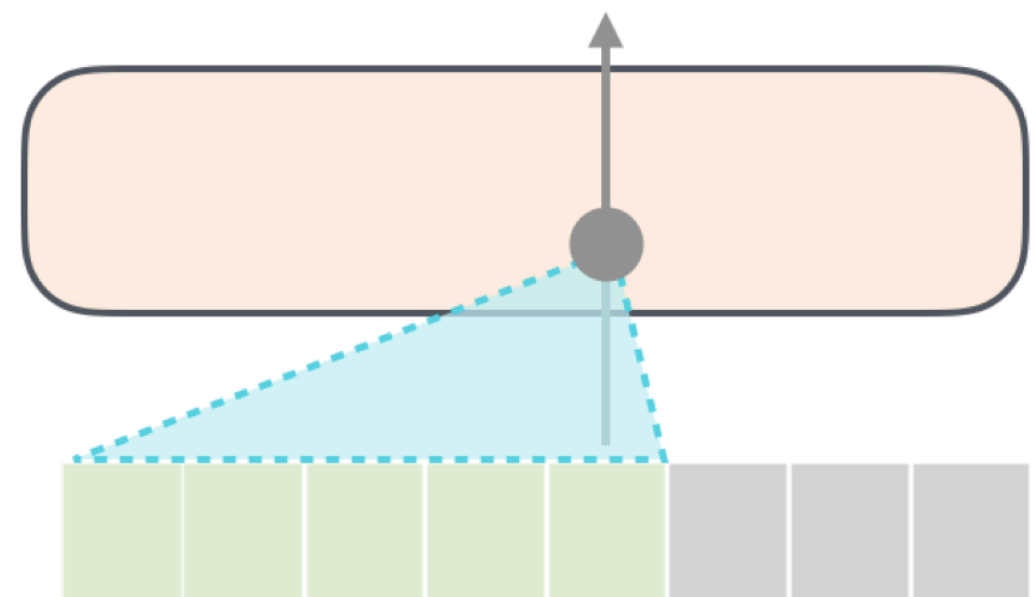
**Encoder models,
e.g. BERT, RoBERTa
(Left and right)**

Self-Attention



**Decoder models,
e.g. GPT
(Left)**

Masked Self-Attention



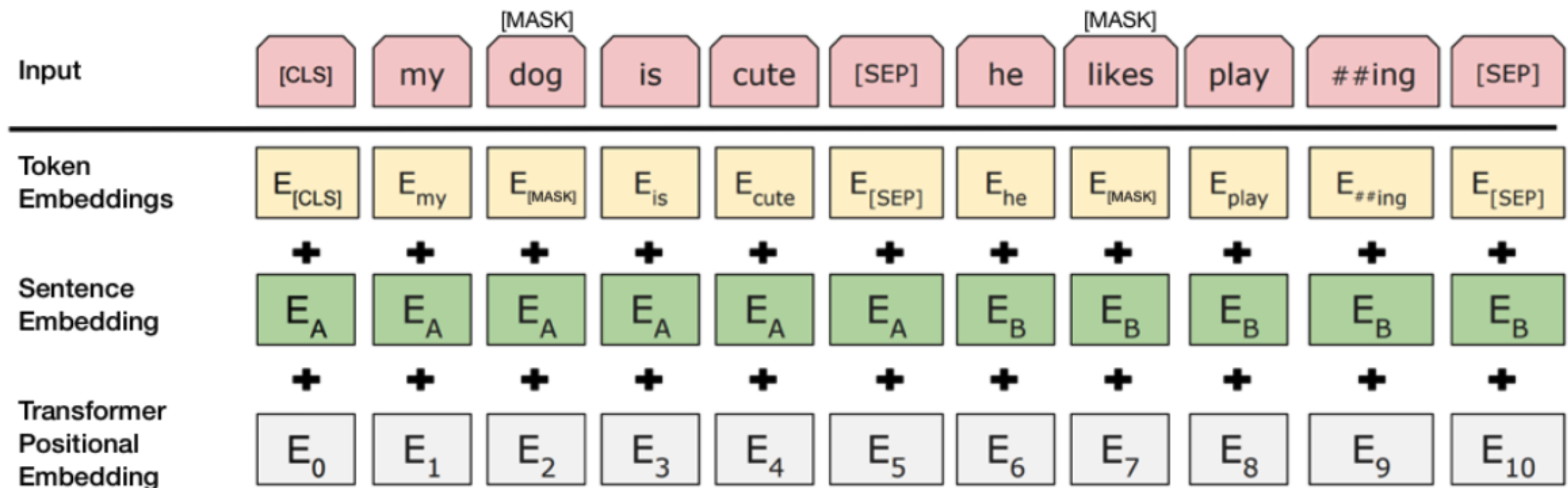
Two types of attention

	Characteristic	LLMs
Encoder-Decoder or Encoder-only (BERT-style)	Training: Masked Language Models Model type: Discriminative Pretrain task: Predict masked words	ELMo [80], BERT [28], RoBERTa [65], DistilBERT [90], BioBERT [57], XLM [54], Xlnet [119], ALBERT [55], ELECTRA [24], T5 [84], GLM [123], XLM-E [20], ST-MoE [133], AlexaTM [95]
Decoder-only (GPT-style)	Training: Autoregressive Language Models Model type: Generative Pretrain task: Predict next word	GPT-3 [16], OPT [126], PaLM [22], BLOOM [92], MT-NLG [93], GLaM [32], Gopher [83], chinchilla [41], LaMDA [102], GPT-J [107], LLaMA [103], GPT-4 [76], BloombergGPT [117]

Yang et al. (2023). Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond

BERT encoder block

- Predict masked words in the sentence
- Next sentence prediction (NSP), sampling positive and negative examples from Wikipedia articles
- Sentence encoding (CLS token) for classification



1,25724351E+00	-4,21386272E-01	2,20382869E-01	-6,18379831E-01
-6,52380407E-01	1,99442339E+00	6,28721058E-01	-7,40208253E-02
1,09820557E+00	8,8621223E-01	1,05409431E+00	1,17599107E-01
1,2035296E+00	-9,11825955E-01	-1,12437144E-01	-2,16738296E+00
-3,73249203E-01	1,36554289E+00	2,58473843E-01	3,04119736E-01
1,81012642E+00	-9,0661788E-01	-1,57513678E+00	-1,120148E-01
7,71892309E-01	1,79286981E+00	7,32205451E-01	6,18414581E-01
-1,1440661E+00	3,6845836E-01	-1,95929229E-01	2,2237692E-02
4,07098472E-01	1,54453969E+00	-1,18874133E+00	-4,51792687E-01
-1,85359195E-01	6,79815412E-01	7,18916833E-01	2,01430583E+00
-3,00920129E-01	-3,40579301E-01	1,07375525E-01	9,08676684E-01
-9,6958673E-01	1,45332611E+00	6,87592089E-01	7,70169273E-02
-7,00117171E-01	5,74636221E-01	1,72431397E+00	4,82968807E-01
9,38793123E-01	1,20942521E+00	-7,3117882E-01	3,96941006E-01
-4,78078574E-01	2,55049884E-01	-5,80259204E-01	2,11581063E+00
1,26037323E+00	3,95636111E-01	9,5505482E-01	-6,04388773E-01
-1,55774248E+00	-5,8375442E-01	-7,72295058E-01	-9,33183968E-01
4,67287451E-01	7,64143229E-01	-2,06774548E-01	-4,14184421E-01
-1,21715939E+00	-7,58659899E-01	-7,82566488E-01	-1,02946937E+00
5,01516834E-03	1,43556499E+00	1,18484712E+00	7,82466769E-01
1,84774891E-01	-1,76700637E-01	1,14600086E+00	1,52692854E+00
-8,7528491E-01	-5,56968749E-01	-5,29614091E-01	1,79106846E-01
-7,87554085E-01	2,25830197E-01	-7,90306628E-02	6,34784222E-01
4,10172969E-01	5,68616316E-02	1,61111736E+00	1,08870912E+00
1,5895848E-01	2,56567502E+00	-6,16940379E-01	3,33929718E-01
-1,70974088E+00	4,2043072E-01	4,77719724E-01	6,42029226E-01
2,68705219E-01	6,84317231E-01	1,12696314E+00	1,24306917E+00
9,74177063E-01	-4,81082976E-01	1,97974074E+00	-1,15706611E+00
-4,01781529E-01	7,81364322E-01	4,13036972E-01	2,10707045E+00
-1,18001306E+00	8,94401968E-01	1,88111103E+00	1,05434859E+00
-7,46218443E-01	-1,38664842E+00	-6,2664181E-01	2,32998788E-01
9,24837232E-01	-9,69367772E-02	-5,01549482E-01	1,03866029E+00
-3,05693603E+00	-3,65617722E-02	1,1940307E-02	-1,40508473E+00
-8,17406997E-02	-3,35481018E-02	-1,23901939E+00	-1,33211744E+00
1,45171678E+00	-1,15909338E+00	2,6075139E+00	1,85218298E+00
4,71510231E-01	4,1262421E-01	3,83586705E-01	-1,1378895E+00
-2,65190101E+00	2,9095E-01	-6,9024086E-01	1,30400836E+00
-2,08187914E+00	-4,8725301E-01	-1,58905828E+00	6,01272225E-01
7,06206143E-01	-2,77167648E-01	9,44868624E-02	-1,0288246E-01

The total number of parameters is 110 million.

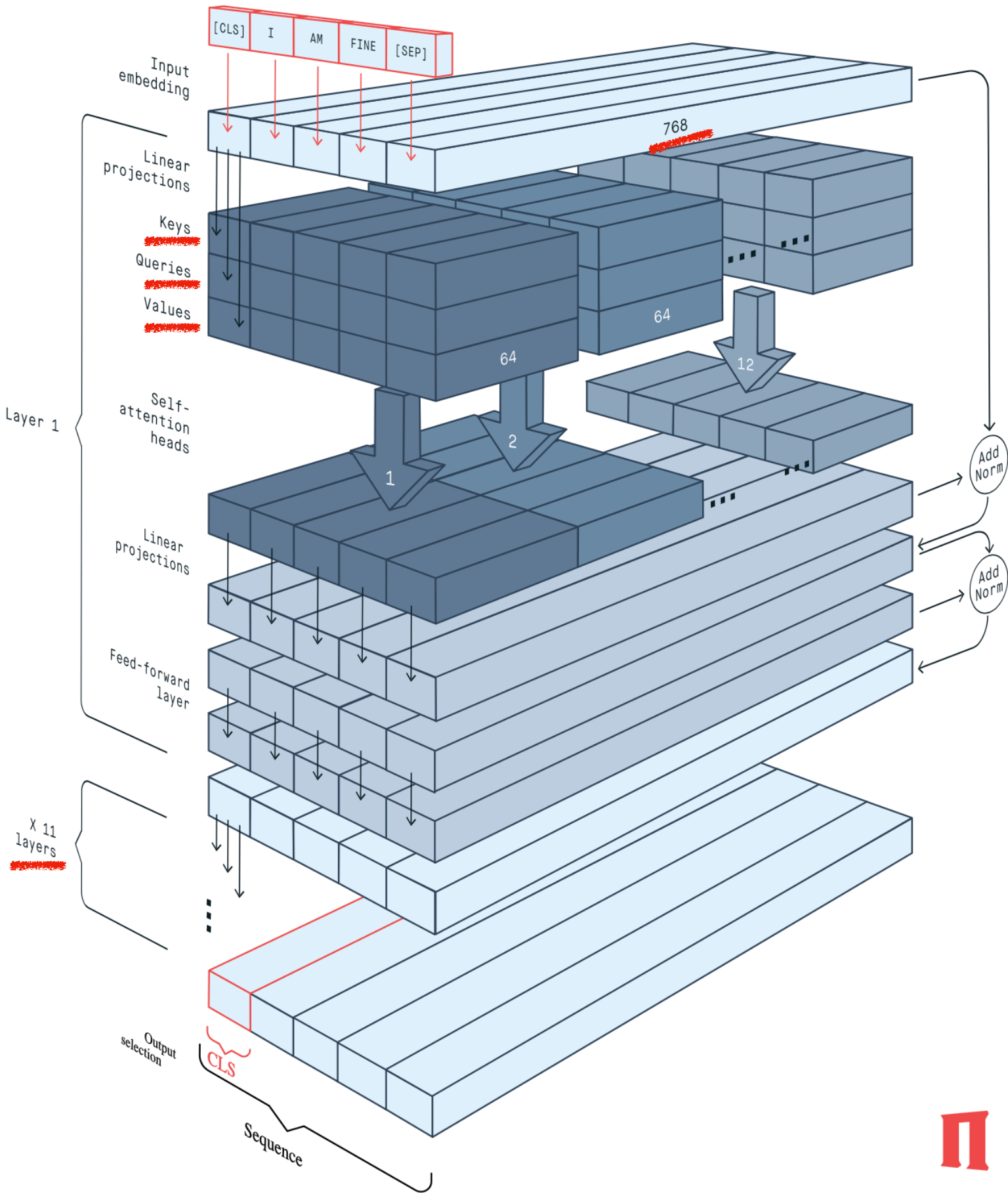
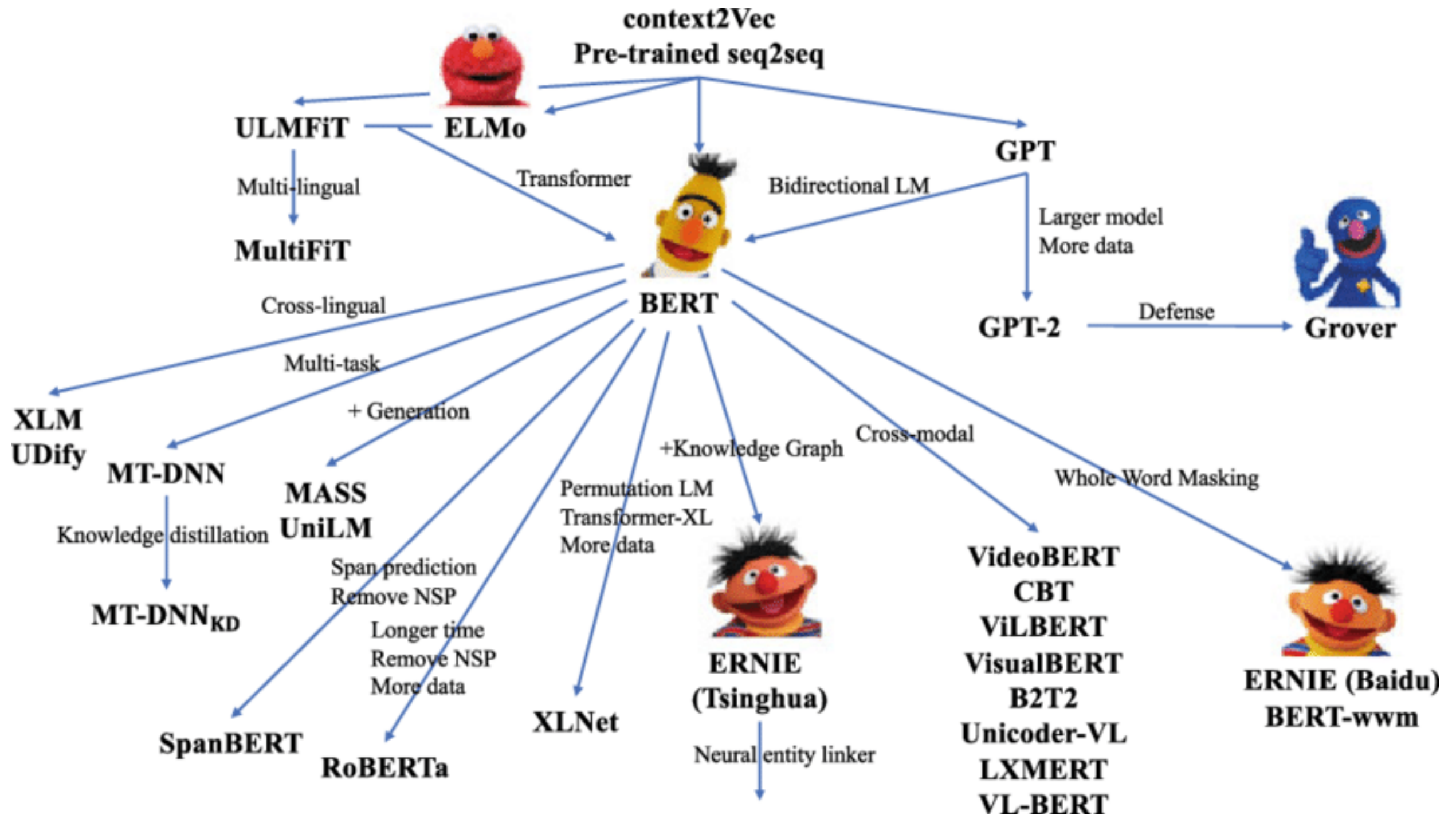


Figure 1. Structure of BERT

BERT's family of language models

<https://huggingface.co>



The evolutionary tree of LLMs

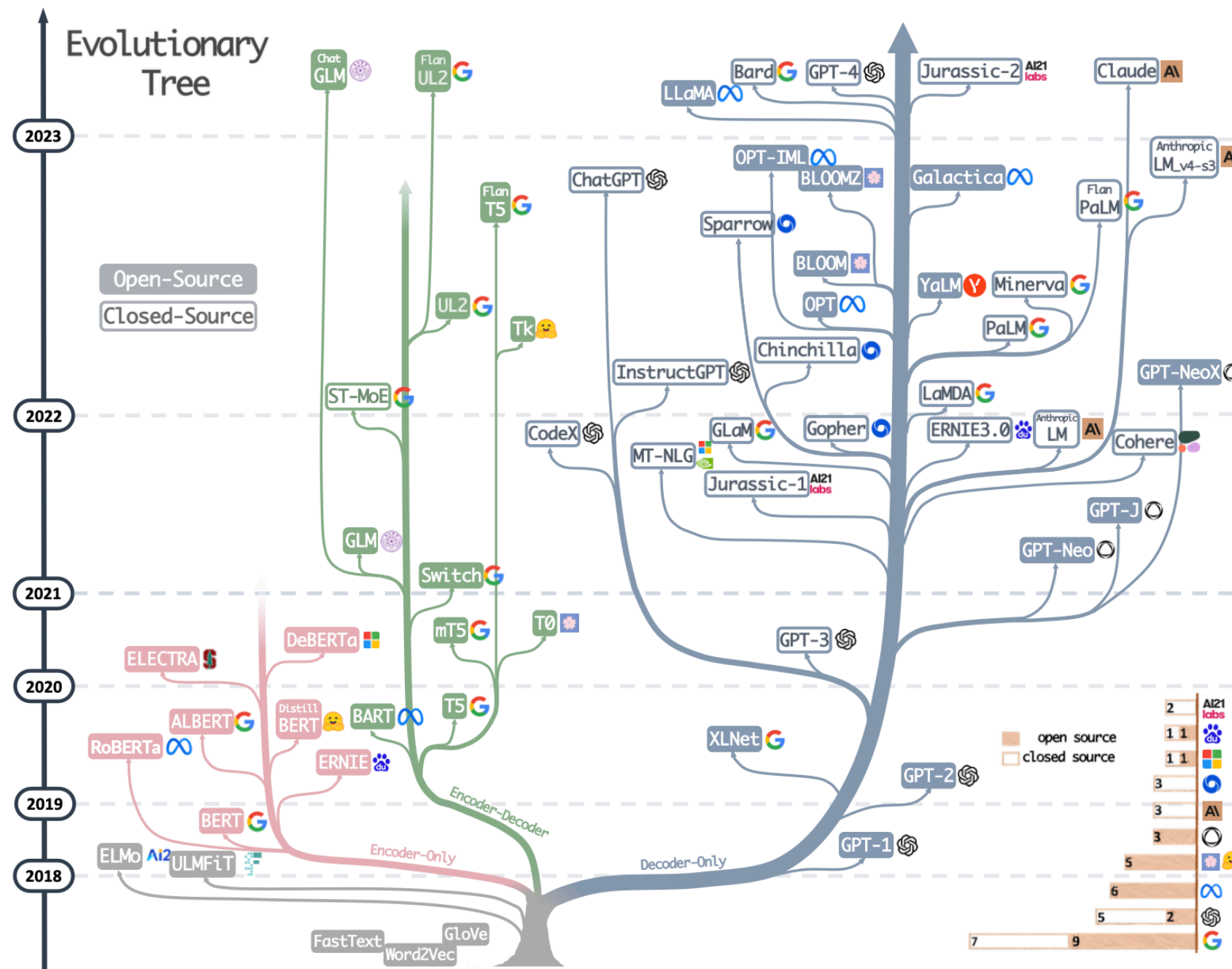
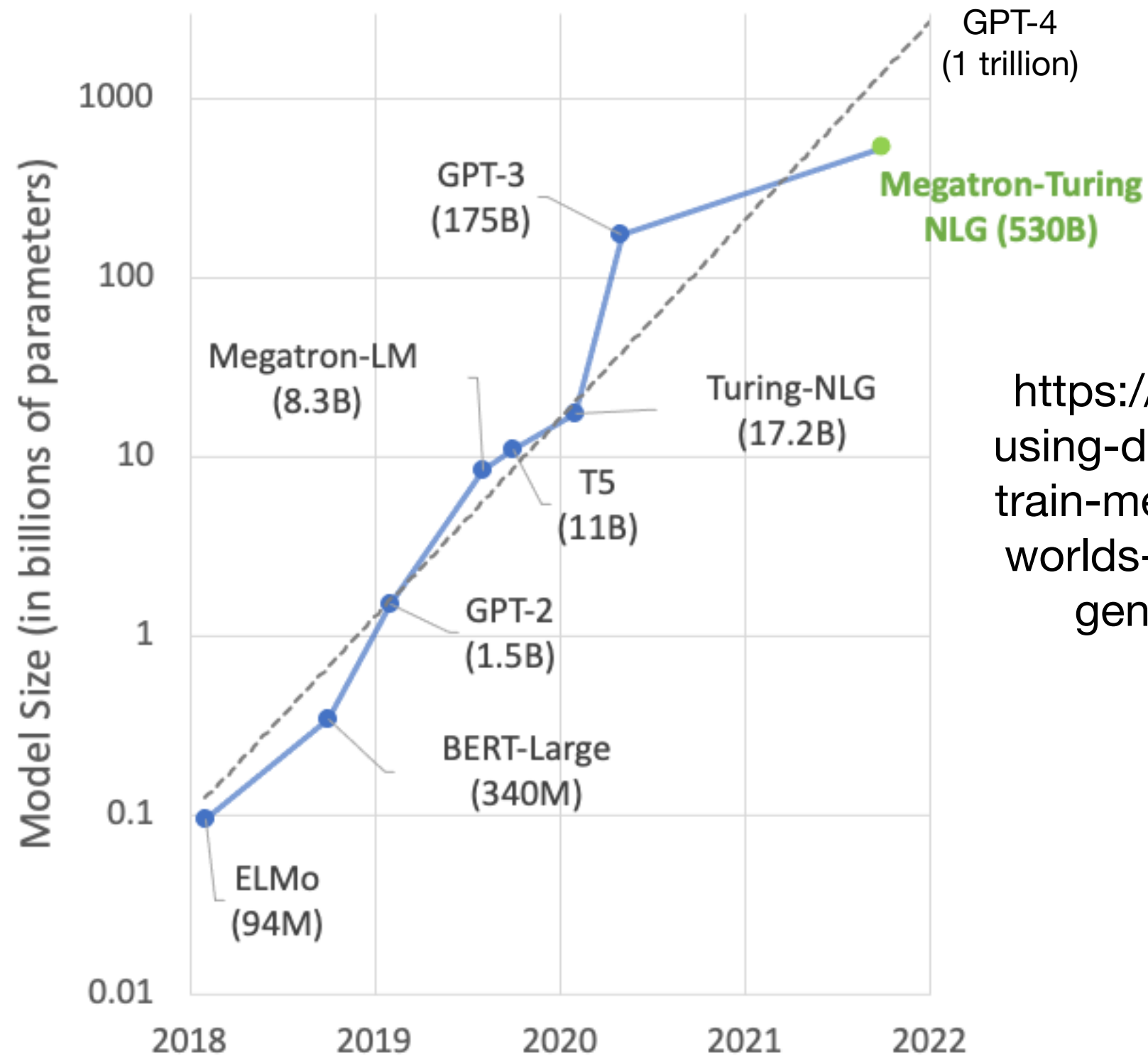


Fig. 1. The evolutionary tree of modern LLMs traces the development of language models in recent years and highlights some of the most well-known models. Models on the same branch have closer relationships. Transformer-based models are shown in non-grey colors: decoder-only models in the blue branch, encoder-only models in the pink branch, and encoder-decoder models in the green branch. The vertical position of the models on the timeline represents their release dates. Open-source models are represented by solid squares, while closed-source models are represented by hollow ones. The stacked bar plot in the bottom right corner shows the number of models from various companies and institutions.

Does size matter?



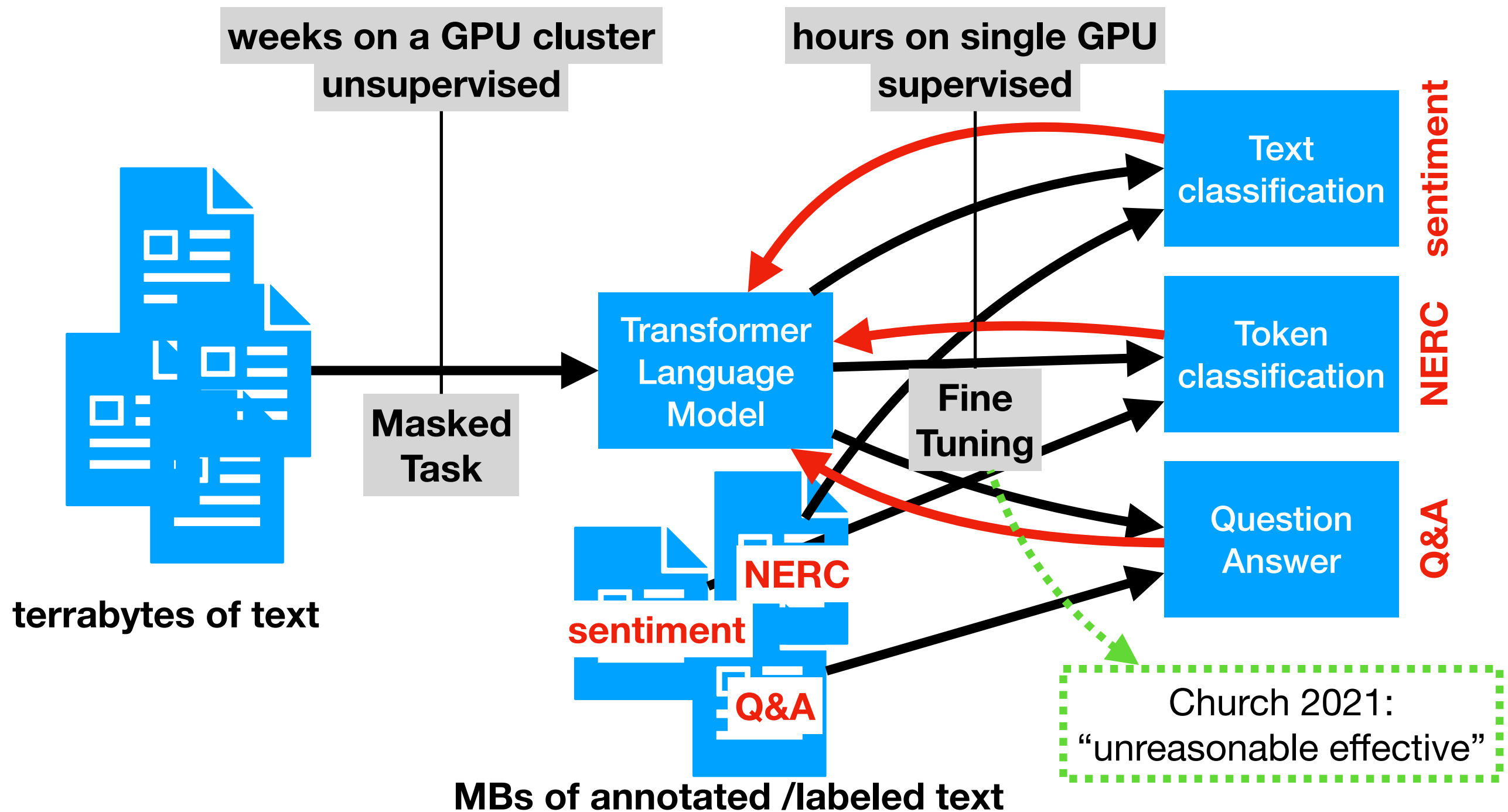
<https://developer.nvidia.com/blog/using-deepspeed-and-megatron-to-train-megatron-turing-nlg-530b-the-worlds-largest-and-most-powerful-generative-language-model/>

Popular data sets for pretraining

Table 2. Some popular datasets for training base models

Dataset	Description
Billion Word Benchmark (Chelba <i>et al.</i> 2013)	A billion words of English
Common Crawl (Buck, Heafield, and van Ooyen 2014)	https://github.com/commoncrawl
Book Corpus (Zhu <i>et al.</i> 2015)	Speech with text
ImageNet (Deng <i>et al.</i> 2009)	14M images, annotated with 21k classes
LibriSpeech (Panayotov <i>et al.</i> 2015)	960 hours of speech with text
LJ Speech	https://keithito.com/LJ-Speech-Dataset/
AISHELL-2 (Du <i>et al.</i> 2018)	https://protect-eu.mimecast.com/s/_t4dC0Vqqsjz5YGswPwKH?domain=aishelltech.com

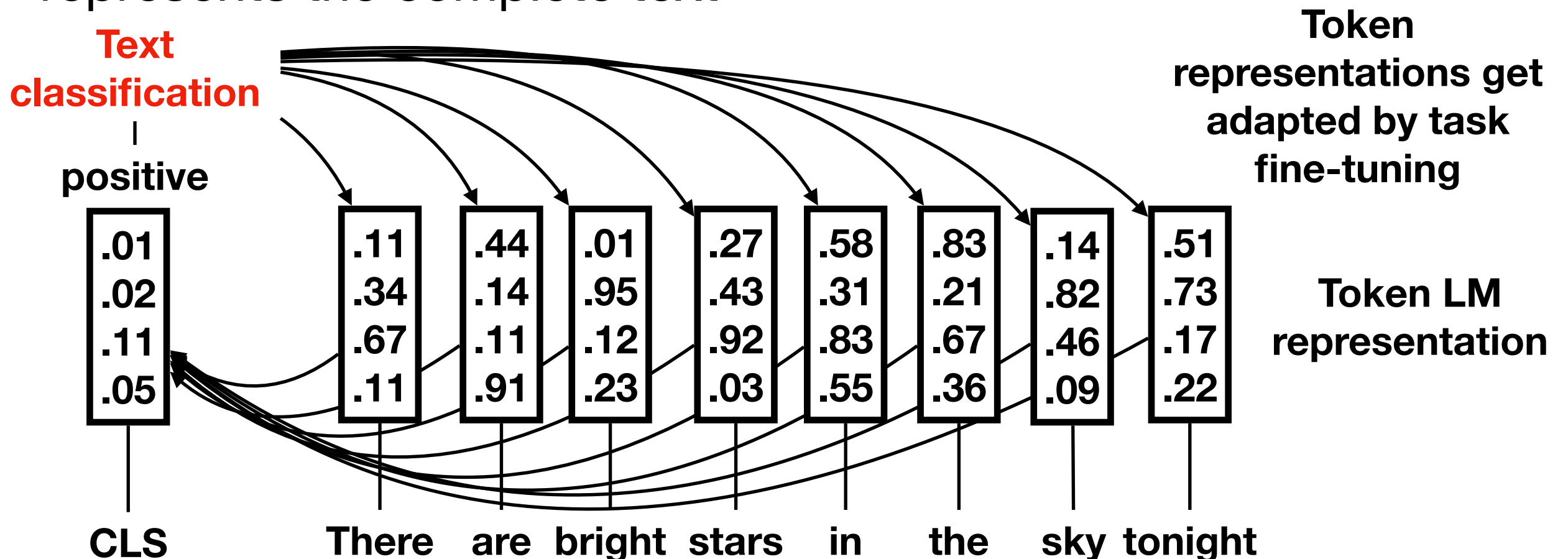
Fine tuning large base/pretrained models



Given 110 million parameters, it learns to predict the label + adjusts the parameters (the internal representation of a transformer is adapted)

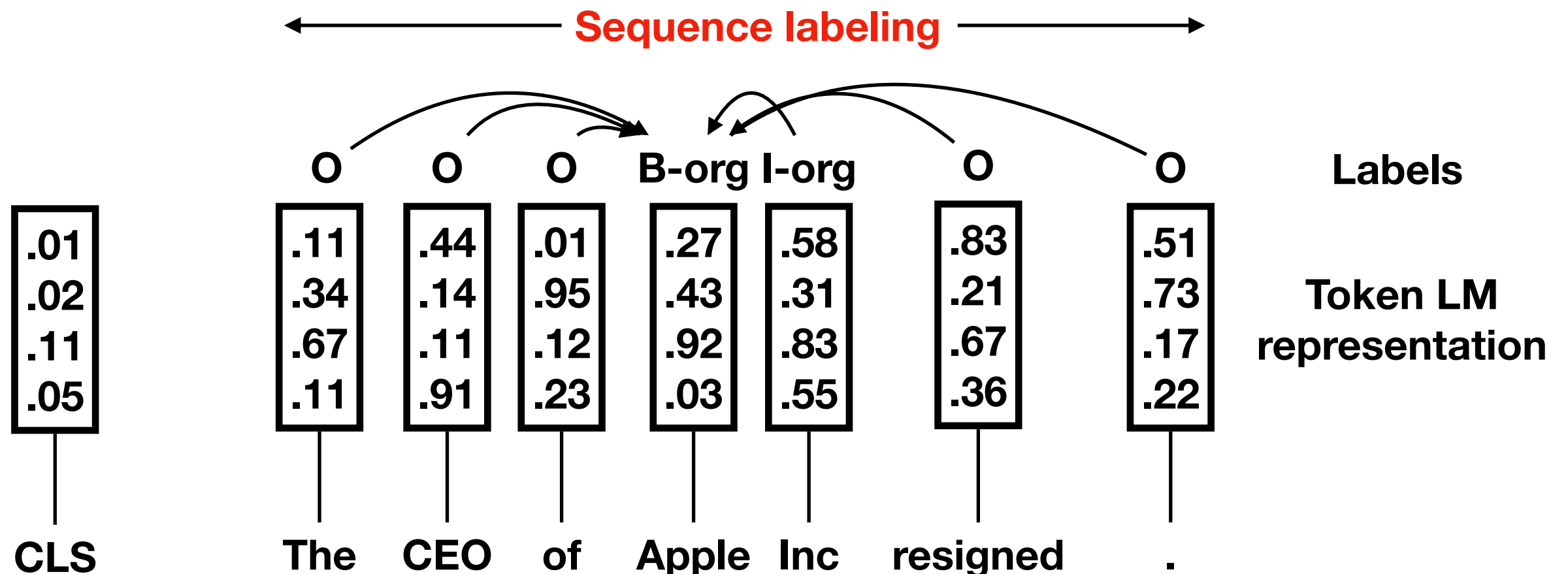
Fine tuning

- Use a language model (LM) to represent a sentence as a sequence of token embeddings
- Train the LM with an additional layer (head) to predict labels either for tokens or for the special CLS token that represents the complete text



Fine tuning

- Use a language model (LM) to represent a sentence as a sequence of token embeddings
- Train the LM to predict labels either for tokens or for the special CLS token that represents the complete text



huggingface.co

- Provides transformer libraries & documentation
- Allows researchers to upload and share trained models on the Model Hub
- Provides an inference API to configure models for specific tasks (aka pipelines)
- <https://huggingface.co>
- <https://simpletransformers.ai> - simple API to use these models